

COURS INTRODUCTION A L'INTELLIGENCE ARTIFICIELLE

UFR STA
LICENCE 2 MPI
SEMESTRE 4

Université Amadou Mahtar Mbow
Prof Amadou Dahirou GUEYE ADG
Année académique 2024-2025

INTRODUCTION A L'INTELLIGENCE ARTIFICIELLE

Plan du cours en détail

Chapitre 1: Introduction à l'IA

Définitions, histoire, domaines d'application, IA symbolique vs connexionniste

Chapitre 2: Représentation de Connaissances en IA

Formalismes de représentation des connaissances, Programmation Logique, Prolog

Chapitre 3: Systèmes experts

Chapitre 4: Introduction au Machine Learning

Apprentissage supervisé / non supervisé

Plan du cours en détail

Chapitre 5: Régression Linéaire

Modèle, fonction coût, descente de gradient

Chapitre 6: Classification

k-NN, régression logistique

Chapitre 7: Réseaux de neurones

Perceptron, neurone formel, introduction à Scikit-learn

Chapitre 8: Evaluation de modèles

Sur-apprentissage, validation croisée, métriques (Précision, Rappel, Courbe ROC, AUC)

Chapitre 9: Découverte sur quelques outils IA

ChatGPT, Leonardo, DeepSeek, etc.

Projet et révision

Chapitre 1: Introduction à l'IA

Plan Chapitre 1

- Définition Sciences des données
- Introduction IA
- Définition IA
- Historique de l'IA
- Les types de l'IA
- Les approches de l'IA
- Fonctionnement de l'IA
- Domaine d'application
- Avantages et inconvénients de l'IA
- Conclusion

C'est quoi la science des données ?

- La **science des données** est un domaine interdisciplinaire qui utilise les mathématiques, les statistiques, le calcul scientifique, les méthodes scientifiques, les process, les algorithmes et les systèmes informatiques automatisés pour extraire et extrapoler des connaissances à partir de grandes quantités de données brutes structurées ou non.
- La **science des données** permet entre autre de produire des algorithmes et méthodes d'analyse de grandes masses de données afin d'en extraire des informations et de réaliser des modèles et prédictions utiles.

C'est quoi la science des données ?

Pour cela, elle se met en œuvre selon les étapes suivantes:

- Bien comprendre et définir l'objectif et problème à résoudre
- Elaborer des stratégies d'analyse des données,
- Recueillir, explorer et analyser les données par la **fouille de données** et **l'analyse exploratoire des données**,
- **Préparer les données** : par le nettoyage et le formatage,
- **Créer des modèles** avec les données à l'aide de différentes méthodes et algorithmes tels:
 - les modèles d'Apprentissage automatique,
 - les modèles d'Apprentissage profond,
- Aider au déploiement et à l'exploitation des modèles et de leurs résultats:
 - sous forme brute
 - sous forme de visuels comme des tableaux de bord
 - sous forme d'application ou d'outils d'aide à la décision.

Source: Wiki

Introduction IA

- L'intelligence est souvent associée à la capacité de raisonnement et de réflexion d'une personne.
- Elle s'oppose à l'instinct, qui correspondrait davantage à un réflexe qu'à une pensée élaborée.
- Depuis l'émergence de la robotique et de l'informatique, les chercheurs essaient d'injecter des notions de l'intelligence humaine dans des machines.
- Étant conçue et fabriquée par l'Homme, on qualifie cette forme d'intelligence d' « artificielle ».

Introduction

- Depuis plusieurs décennies, notre monde a connu de nombreuses avancées technologiques, mais l'une d'entre elles retient particulièrement notre attention : l'intelligence artificielle (IA).
- Des scientifiques comme Alan Turing, John McCarthy ou encore Marvin Minsky ont grandement contribué à son émergence.
- Aujourd'hui, l'IA se manifeste dans de nombreux domaines, notamment dans les jeux vidéo, mais aussi dans des secteurs clés tels que la médecine, la défense ou encore les transports, etc.

Définition

L'intelligence artificielle désigne l'ensemble des techniques permettant à des machines d'**imiter** certaines fonctions cognitives humaines, telles que: la perception, le raisonnement, l'apprentissage, la prise de décision, l'interaction avec l'environnement.

4 types de définitions

Système qui **pense**
comme des **humains**

Système qui **pense**
rationnellement

Système qui **agit**
comme des **humains**

Système qui **agit**
rationnellement

Penser comme les humains : Approche cognitive

- ❑ Requier des théories scientifiques de l'activité interne du cerveau.
- ❑ Implémenter les théories et comparer avec les humains.
- ❑ Il faut prédire et tester le comportement de sujets humains (sciences cognitives)
- ❑ Les sciences cognitives combinent modèles informatiques de l'IA et techniques expérimentales de psychologie pour élaborer des théories précises et vérifiables du fonctionnement de l'esprit.

Penser rationnellement

- ❑ Lois de la pensée données par la logique (Aristote, Leibniz, Boole, Frege...).
- ❑ *Socrate est un homme; tous les hommes sont mortels; donc Socrate est mortel.*
- ❑ La logique formelle permet d'écrire des énoncés sur les objets dans le monde.
- ❑ Lien direct entre les mathématiques et la philosophie vers l'IA moderne.
- ❑ Il est difficile de traduire les connaissances et les états du monde réel en des équations logiques (incertitude).

Agir rationnellement

- ❑ Agir selon ses croyances pour atteindre des objectifs

 - Exemple: *réflexe de cligner les yeux, retirer sa main d'un objet brûlant...*

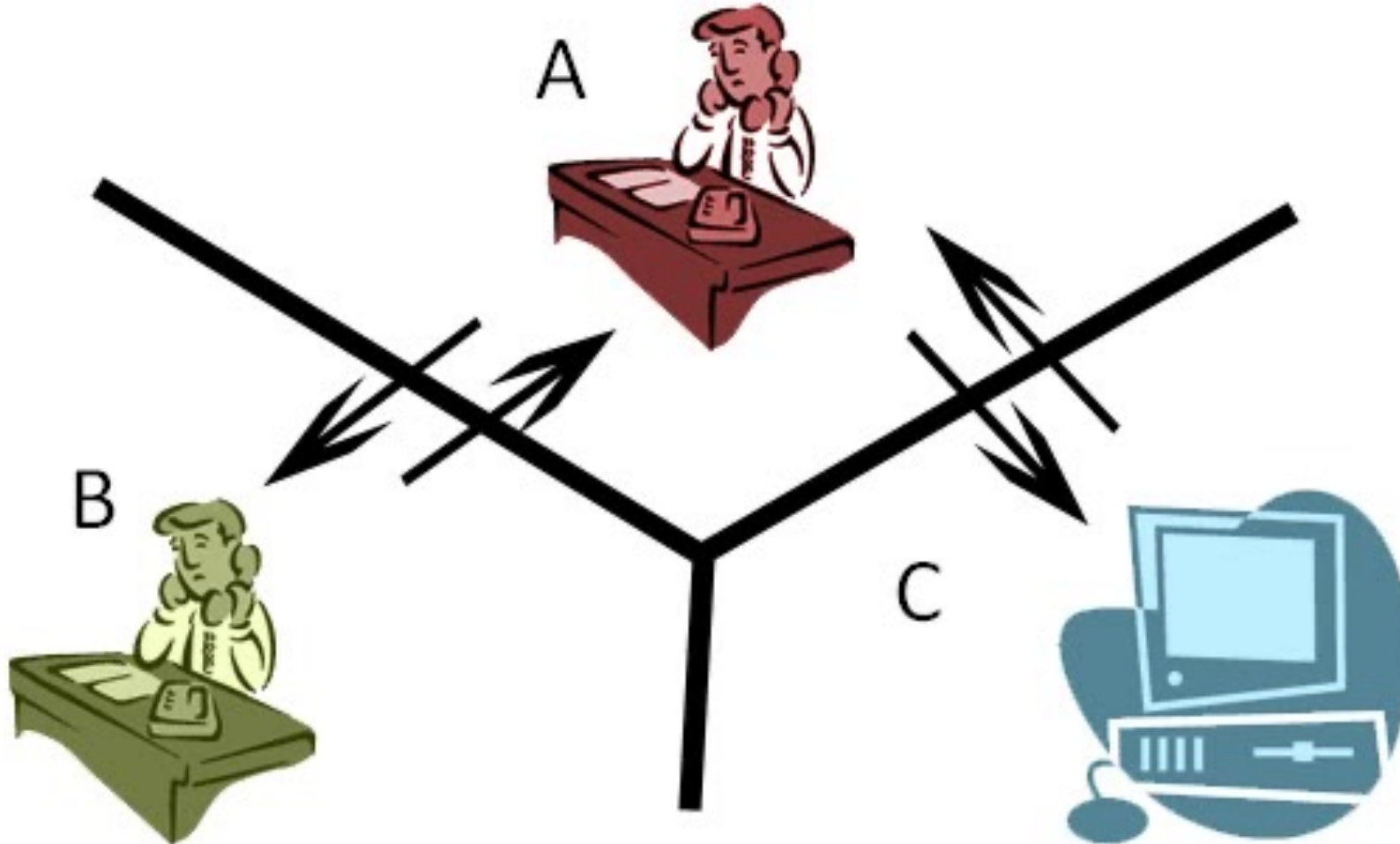
- ❑ Approche reposant sur la notion d'agent

 - Agent rationnel:** Agent qui agit de manière à atteindre la meilleure solution ou, dans un environnement incertain, la meilleure solution prévisible.

Agir comme les humains : Test de Turing

- ❑ 1950
- ❑ Inspiré du « jeu de l'imitation »
- ❑ Un interrogateur humain dialoguant avec deux entités (sans les voir) doit déterminer laquelle est l'humain et laquelle est la machine. S'il se trompe plus souvent que quand il a à distinguer, dans les mêmes circonstances, une femme d'un homme, alors la machine franchit le test.

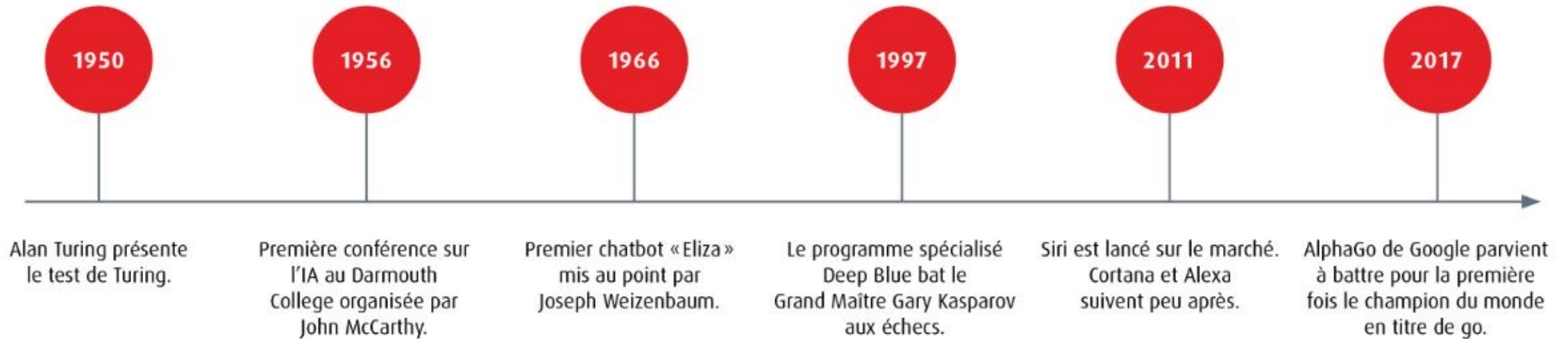
Test de Turing



Historique de l'IA

- Le test de Turing est une proposition de test d'intelligence artificielle fondée sur la faculté d'une machine à imiter la conversation humaine.
- Décrit par Alan Turing en 1950 dans sa publication *Computing Machinery and Intelligence*, ce test consiste à mettre un humain en confrontation verbale à l'aveugle avec un ordinateur et un autre humain.

Historique de l'IA



Chronologie

Objectifs et ambitions de l'IA

Objectif principal :

Créer des systèmes capables de simuler l'intelligence humaine pour résoudre des problèmes complexes de manière autonome.

Objectifs et ambitions de l'IA

Objectifs spécifiques :

- Automatiser les tâches cognitives : raisonnement, prise de décision, apprentissage.
- Comprendre le langage humain (NLP).
- Percevoir l'environnement : via la vision, l'audition, etc.
- Apprendre à partir de données : classification, prédiction, détection d'anomalies.
- Interagir avec le monde réel : via des robots ou des agents logiciels.
- S'adapter et évoluer avec l'expérience.

Objectifs et ambitions de l'IA

Court terme

- Améliorer l'efficacité des services (santé, finance, industrie...)
- Automatiser les tâches répétitives ou dangereuses
- Offrir des assistants intelligents (chatbots, traducteurs, etc.)

Objectifs et ambitions de l'IA

Moyen terme

- IA généraliste capable de transférer des compétences d'un domaine à un autre.
- Voitures autonomes, assistants médicaux, etc.

Long terme

- Atteindre une IA forte ou superintelligence.
- Créer des systèmes avec raisonnement, conscience et intuition

Les types d'IA

L'intelligence artificielle se divise en deux grandes catégories selon son niveau de complexité et d'autonomie cognitive :

- l'IA faible (ou étroite)
 - l'IA forte (ou générale).
-
- L'IA super intelligent
 - L'IA hybride

Les types d'IA

IA Faible

- L'IA faible est une intelligence artificielle spécialisée dans une seule tâche.
- Elle est conçue pour simuler une capacité humaine précise (par exemple reconnaître des visages ou comprendre une commande vocale), sans conscience ni compréhension réelle.
- C'est la forme d'IA la plus répandue aujourd'hui.
- Elle fonctionne avec des règles, des algorithmes d'apprentissage ou des modèles statistiques adaptés à une tâche bien définie

Les types d'IA

IA Faible

Exemples : Siri, Google Translate, moteurs de recommandation, chatbots, systèmes de reconnaissance d'image.



Les types d'IA

IA Forte

- L'IA forte vise à reproduire une intelligence comparable à celle des humains.
- Elle serait capable de raisonner, comprendre, apprendre et s'adapter à des contextes nouveaux, comme le ferait un être humain.
- Une telle IA pourrait accomplir n'importe quelle tâche intellectuelle, indépendamment du domaine.

Les types d'IA

Caractéristique	IA faible (Narrow AI)	IA forte (Strong AI)
Définition	Systèmes conçus pour accomplir une tâche spécifique	Systèmes capables de raisonner, apprendre et comprendre comme un humain
Capacité	Limitée à un domaine précis	Polyvalente et autonome
Exemples	Siri, Google Translate, voitures autonomes	Théorique : robot doté d'une conscience, d'une compréhension globale
Niveau d'intelligence	Simule l'intelligence pour une tâche	Possède une intelligence comparable à celle des humains
État actuel	Déjà déployée dans de nombreuses applications	Encore à l'état de recherche, non réalisée

Les approches de l'IA

Approche symbolique (ou logique)

L'approche symbolique, également appelée IA classique ou logique, repose sur la manipulation de symboles et de règles logiques pour représenter les connaissances et résoudre des problèmes.

Les approches de l'IA

Approche symbolique (ou logique)

L'approche symbolique, également appelée IA classique ou logique, repose sur la manipulation de symboles et de règles logiques pour représenter les connaissances et résoudre des problèmes.

Les approches de l'IA

Approche symbolique :

Exemples d'outils et techniques

- Systèmes experts
- Moteurs d'inférence (à partir des faits, le système prend des décisions: chainage avt ou ar. Ex: vol $\rightarrow$ prison)
- Logique propositionnelle (simple proposition) et logique du premier ordre (prédicat et quantificateur)
- Représentation des connaissances (ontologies, graphes sémantiques). Ici on cherche à trouver des liens entre les données?

Les approches de l'IA

Approche symbolique :

Les systèmes experts sont utilisés dans de nombreux domaines. En voici quelques exemples :

- des [moteurs de recherche](#) sur Internet, qui permettent à partir d'une requête, de trouver les pages Web correspondantes ;
- des outils spécialisés pour aider un médecin à diagnostiquer une maladie ;
- des outils utilisés dans l'aide au diagnostic des pannes automobiles ;
- des outils spécialisés pour aider un ingénieur à concevoir un produit. Les premiers ont été développés par l'[IBM](#) et baptisés [Dendral](#), puis [Mycin](#), etc. ;
- des programmes destinés aux joueurs d'[échecs](#) ou de [go](#) (comme le programme [Deep Blue](#)), qui peuvent analyser les positions du jeu et proposer le meilleur coup possible pour le joueur ou l'ordinateur^[4] ;
- des outils destinés à l'[aide à la décision](#) pour les entreprises ;
- des systèmes de recommandation (pour les achats en ligne par exemple) utilisant la [logique floue](#) et un chaînage arrière^[5] ;
- des systèmes permettant la reconnaissance vocale ;
- des systèmes permettant la traduction automatique ;
- des systèmes permettant le traitement du langage naturel ;
- etc.

Les approches de l'IA

Approche connexionniste

- L'approche connexionniste cherche à imiter le fonctionnement du cerveau humain en utilisant des réseaux de neurones artificiels.
- Elle repose sur l'idée que l'intelligence émerge de l'interconnexion d'unités simples (les neurones) capables d'apprendre à partir de données.

Les approches de l'IA

Approche statistique

- L'approche statistique repose sur l'analyse mathématique des données pour modéliser des relations, faire des prédictions ou prendre des décisions.
- Elle considère que l'intelligence peut émerger de la détection de motifs dans de grands ensembles de données.

Les approches de l'IA

Approche statistique

Techniques courantes

- Régression linéaire et logistique
- Méthodes de classification : k-NN, SVM, arbres de décision
- Méthodes de clustering : k-means
- Méthodes d'ensemble : random forest, boosting

Fonctionnement de l'IA

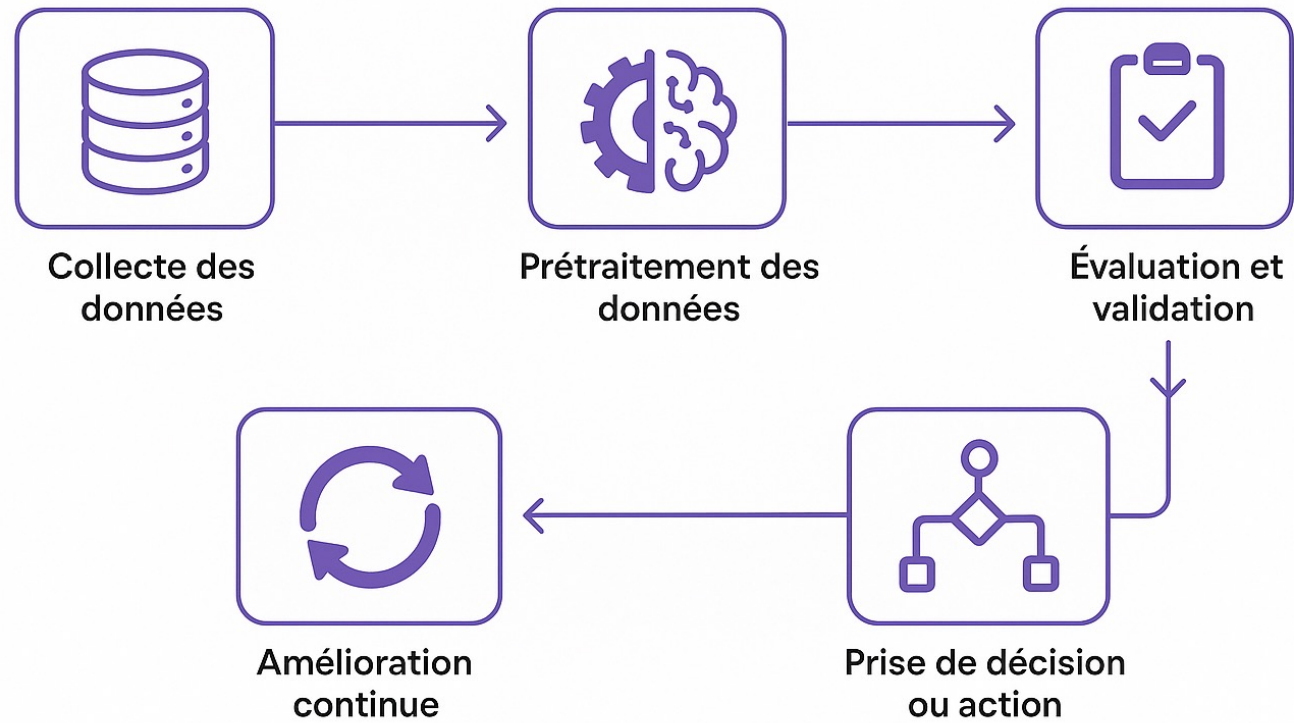
- Le fonctionnement de l'intelligence artificielle repose sur la capacité des machines à traiter des données, apprendre à partir de ces données, puis prendre des décisions ou effectuer des actions.
- Voici les grandes étapes du fonctionnement typique d'un système d'IA :

Fonctionnement de l'IA

- Collecte des données
- Prétraitement des données
- Apprentissage (training)
- Évaluation et validation
- Prise de décision ou action
- Amélioration continue

Fonctionnement de l'IA

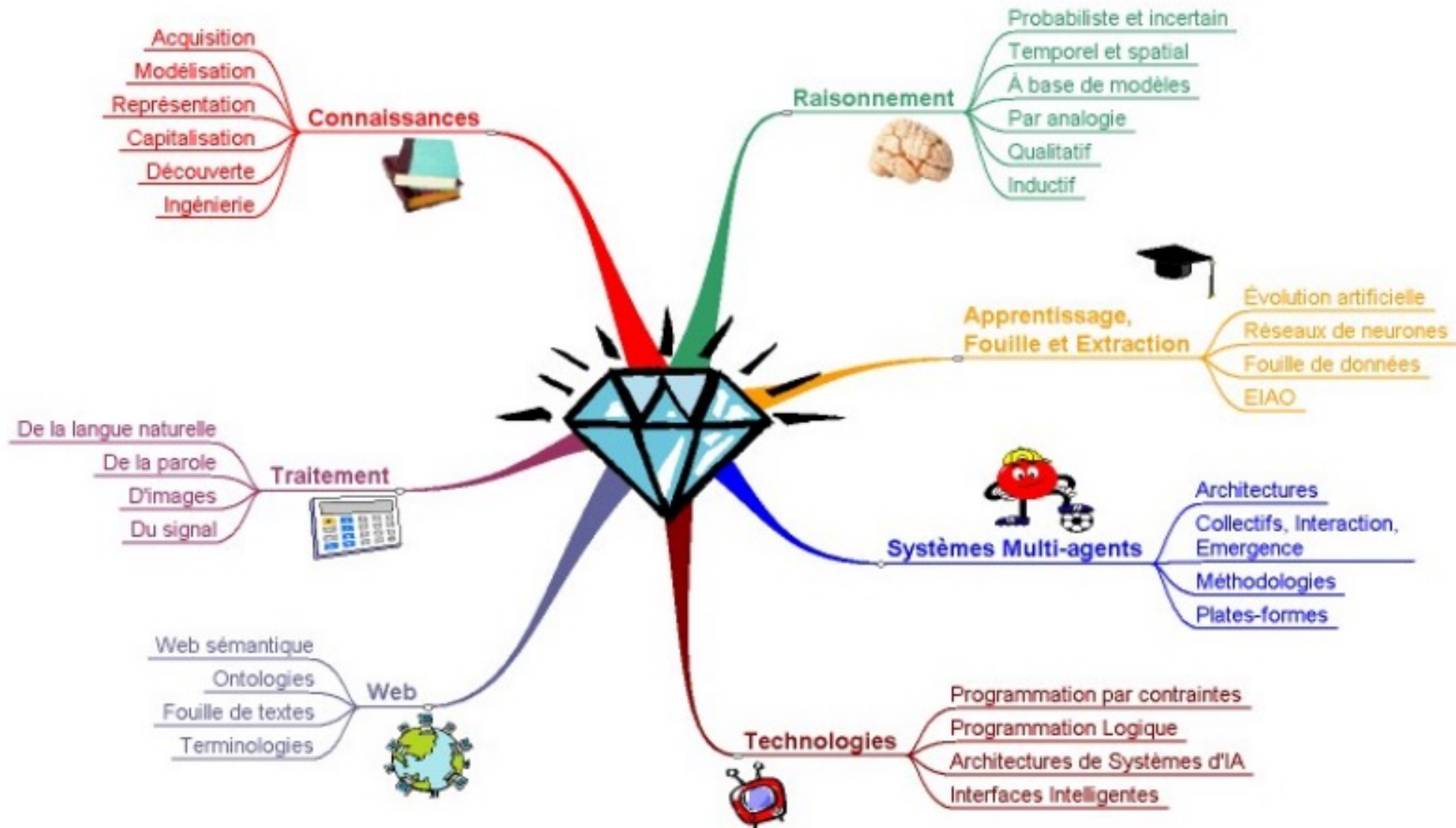
Fonctionnement de l'IA



Les domaines d'application

- Vision par ordinateur : reconnaissance faciale, imagerie médicale.
- Traitement du langage naturel : assistants vocaux, traduction automatique.
- Robotique : robots industriels, véhicules autonomes.
- Médecine : diagnostic assisté, chirurgie robotisée.
- Finance : analyse de risques, détection de fraudes.
- Éducation, agriculture, sécurité, logistique, etc.

Domaines d'application



Avantages de l'IA

- Automatisation des tâches répétitives.
- Gain de temps, réduction des erreurs humaines.
- Aide à la prise de décision rapide et efficace.
- Accessibilité des services (santé, éducation...).

Inconvénient de l'IA

- Risque de chômage technologique.
- Dépendance excessive aux machines.
- Problèmes éthiques (biais, vie privée, décisions non transparentes).
- Détournement malveillant (ex : IA dans les armes).

Avantages et inconvénients de l'IA

Avantages de l'IA



Automatisation des tâches répétitives

Gain de temps, réduction des erreurs humaines



Amélioration de la productivité

Accélération des processus dans l'industrie, les services et la recherche



Précision et efficacité

Dagnostic médical, reconnaissance faciale, prévisions météo, etc.

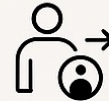


Disponibilité 24h/24

Personnalisation des services
Recommandations sur Netflix, Amazon, Google, etc.

Aide à la décision

Inconvénients de l'IA



Perte d'emplois humains

Remplacement de certains métiers par des machines



Biais algorithmiques

L'IA peut reproduire ou amplifier les discriminations contenues dans les données



Dépendance technologique

Risque de perte de compétences humaines et d'autonomie



Atteinte à la vie privée

Collecte massive de données personnelles



Coût de développement élevé

Accès limité pour certains pays ou petites entreprises

Conclusion

L'intelligence artificielle est un domaine en pleine expansion qui influence profondément notre quotidien. Mieux la comprendre, c'est mieux anticiper ses usages, ses limites et ses implications. Elle représente à la fois une opportunité technologique et un enjeu éthique pour les générations futures.

Conclusion

- Dans cette perspective, la logique propositionnelle et la logique du premier ordre jouent un rôle fondamental : elles constituent les bases formelles sur lesquelles reposent de nombreux systèmes d'IA, notamment en raisonnement automatisé, vérification de programmes et traitement du langage naturel.
- Maîtriser ces outils logiques permet non seulement de mieux concevoir des systèmes intelligents, mais aussi de mieux comprendre leurs mécanismes de déduction, leurs capacités d'inférence et leurs limites rationnelles.

Chapitre 2:

Représentation de connaissances en Intelligence Artificielle

SOMMAIRE

- ☐ Introduction
- ☐ Donnée VS Connaissance VS information
- ☐ Représentation de connaissances
- ☐ Formalismes de représentation de connaissances

Introduction

- ❑ La façon de représenter les connaissances peut faire en sorte que la résolution d'un problème soit plus ou moins facile.

Donnée, Information, Connaissance

- **Donnée (data):** Une donnée est le résultat brut d'une observation ou d'une mesure. Elle est généralement dépourvue de contexte ou d'interprétation, ce qui limite son utilité immédiate pour la prise de décision. Exemples de différentes sources de données brutes
 - **Le texte :** Il s'agit de données non structurées provenant de sources telles que des livres, des documents et des courriels.
 - **Les chiffres :** Les données numériques brutes peuvent provenir d'enquêtes, d'expériences ou d'autres sources.
 - **Images :** Les images brutes sont utilisées dans des domaines tels que l'apprentissage automatique et la vision par ordinateur.
 - **Audio :** Les données audio pertinentes dans des domaines tels que la reconnaissance vocale et la recherche musicale doivent être traitées pour extraire et utiliser des informations précieuses.
- **Information (information):** Une information est une donnée à laquelle on a donné un sens ou une interprétation dans un contexte donné. Elle devient ainsi compréhensible et utilisable.
- **Connaissance (Knowledge):** La connaissance est le fruit d'une analyse approfondie des informations. Elle repose sur l'expérience, l'étude ou la pratique, et permet d'agir, de décider ou d'anticiper.

Représentation de connaissances

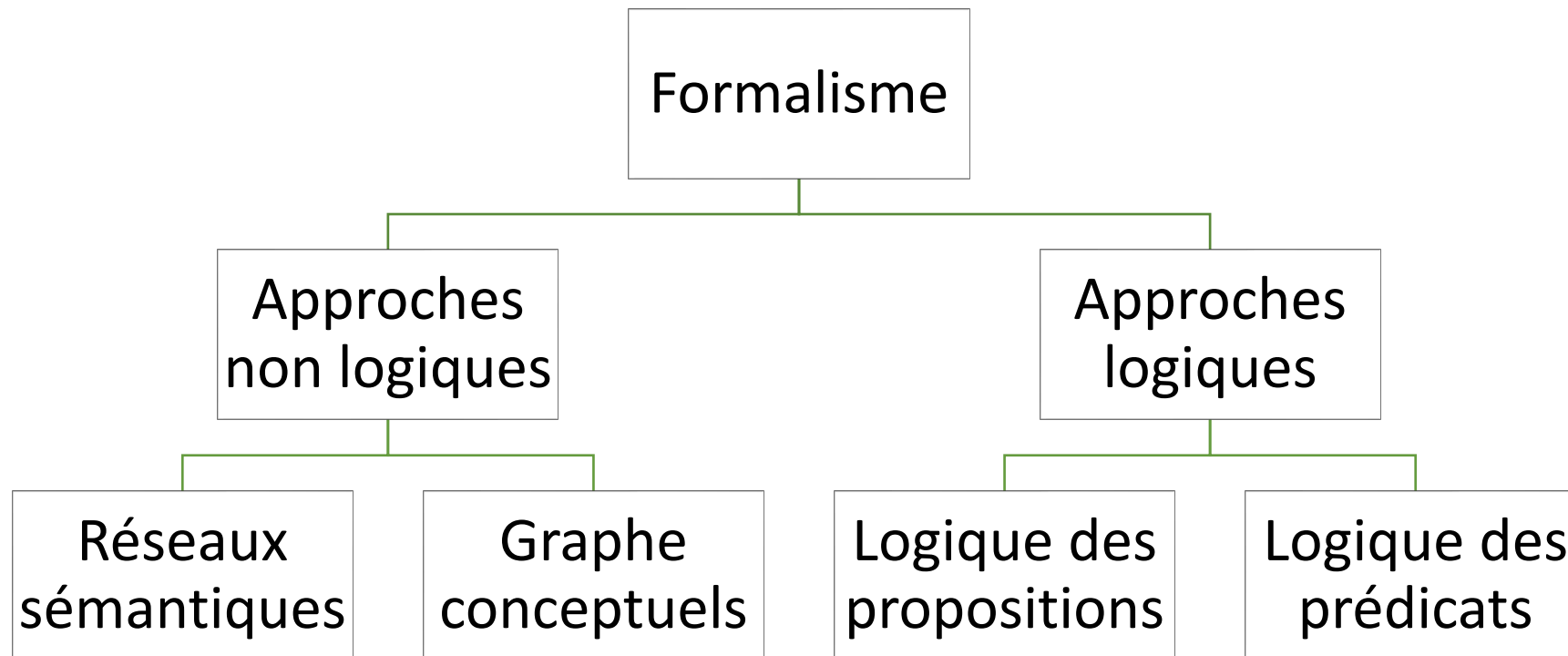
Knowledge Representation

- ❑ La représentation des connaissances est une branche de l'IA qui étudie comment modéliser les connaissances humaines de façon formelle afin qu'elles soient compréhensibles et utilisables par une machine.
- ❑ Elle permet à une machine de stocker, manipuler et raisonner sur des faits, des concepts, des objets et des relations.

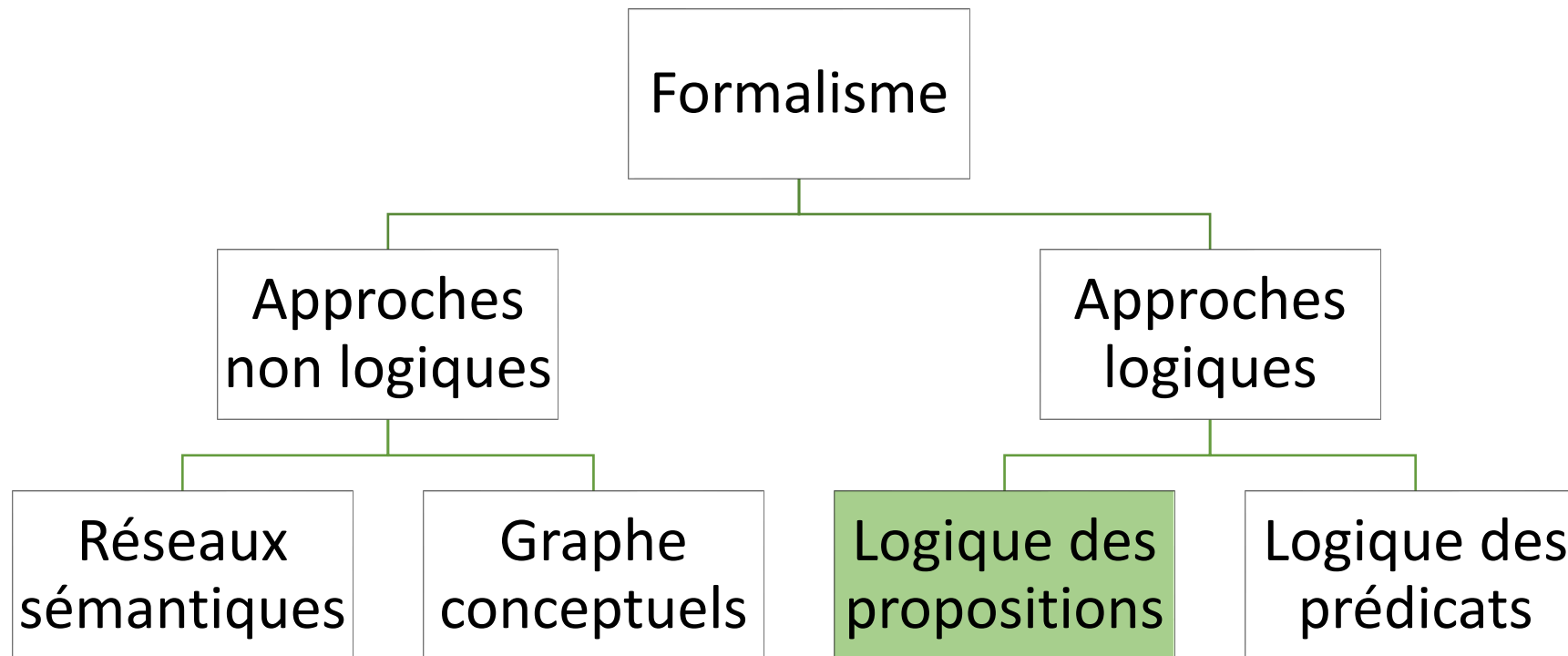
Formalismes de représentation de connaissance

- ❑ Les connaissances concernent des faits, considérés vrais dans un certain monde.
- ❑ Pour représenter ces faits, on a recours à un formalisme ou mode de représentation.
- ❑ Au niveau des faits, on traitera des objets et les relations qu'ils entretiennent.
- ❑ Au niveau du formalisme, on définira des symboles et des opérations sur ces symboles.
- ❑ Le langage naturel est un outil très informel pour représenter les connaissances.
- ❑ L'arithmétique et la logique sont des outils plus formels pour la représentation de connaissance.

Formalismes de représentation de connaissance



Formalismes de représentation de connaissance



Approches non logiques

Logique des propositions

- ❑ Langage formel constitué d'une syntaxe et d'une sémantique permettant de représenter des **connaissances** à partir desquelles nous pouvons tirer des **conclusions**.
- ❑ La syntaxe décrit l'ensemble des **formules** qui **appartiennent** au **langage**.
- ❑ La sémantique permet de donner un **sens** aux **formules** du **langage**.
- ❑ Le vocabulaire de la logique des propositions est constitué :
 - De **propositions** que l'on désignera par les lettres minuscules de l'alphabet $a, b, c, d, \dots, z$
 - De **connecteurs logiques** ($\neg, \vee, \wedge, \rightarrow, \leftrightarrow$),
La priorité en ordre décroissant : ($\neg, \wedge, \vee, \rightarrow, \leftrightarrow$)
 - De **parenthèses**

Approches non logiques

Logique des propositions

connecteurs logiques ($\neg, \vee, \wedge, \rightarrow, \leftrightarrow$),

Symbole	Nom	Lecture/Signification	Exemple (avec p et q)
$\neg$	Négation	« non », inverse la valeur de vérité	$\neg p$: « non p »
$\vee$	Disjonction	« ou » logique (au moins une vraie)	$p \vee q$: « p ou q »
$\wedge$	Conjonction	« et » logique (vrai si les deux sont vraies)	$p \wedge q$: « p et q »
$\rightarrow$	Implication	« si... alors... » (condition suffisante)	$p \rightarrow q$: « si p alors q »
$\leftrightarrow$	Équivalence	« si et seulement si » (condition nécessaire et suffisante)	$p \leftrightarrow q$: « p si et seulement si q »

Logique des propositions

On considère les propositions suivantes :

- Si Pierre est rentré chez lui, alors Jean est allé au cinéma.
- Marie est à la bibliothèque ou Pierre est rentré chez lui.
- Si Jean est allé au cinéma, alors Marie est à la bibliothèque ou Pierre est rentré chez lui.
- Marie n'est pas à la bibliothèque et Jean est allé au cinéma.
- Pierre est rentré chez lui.

Logique des propositions

Notons:

- **M** la proposition Marie est à la bibliothèque
- **J** la proposition Jean est allé au cinéma
- **P** la proposition Pierre est rentré chez lui

- $P \Rightarrow J$
- $M \vee P$
- $J \Rightarrow (M \vee P)$
- $\neg M \wedge J$
- P

Logique des propositions

- Cinq règles pour évaluer un énoncé en fonction d'un modèle **m** :
- $\neg P$ est **vrai** ssi **P** est **faux**
 - $P \wedge Q$ est **vrai** ssi **P** est **vrai** **ET** **Q** est **vrai**
 - $P \vee Q$ est **vrai** ssi **P** est **vrai** **OU** **Q** est **vrai**
 - $P \Rightarrow Q$ est **vrai** sauf si
P est **vrai** **ET** **Q** est **faux**
 - $P \Leftrightarrow Q$ est **vrai** ssi **P** est **vrai** **ET** **Q** est **vrai**

Logique propositionnelle

Sémantique

Table de vérité

Les tables de vérité montrent comment la valeur d'une formule dépend de celles de ses composantes.
Exemples de tables : Négation ($\neg P$)

P	$\neg P$
V	F
F	V

Logique propositionnelle

Sémantique

Table de vérité

Les tables de vérité montrent comment la valeur d'une formule dépend de celles de ses composantes.

Exemples de tables : **Négation ($\neg P$)**

P	$\neg P$
V	F
F	V

Logique propositionnelle

Sémantique

Conjonction ($P \wedge Q$)

P	Q	$P \wedge Q$
V	V	V
V	F	F
F	V	F
F	F	F

Logique propositionnelle

Sémantique

Disjonction ($P \vee Q$)

P	Q	$P \vee Q$
V	V	V
V	F	V
F	V	V
F	F	F

Logique propositionnelle

Sémantique

Implication ($P \rightarrow Q$)

P	Q	$P \rightarrow Q$
V	V	V
V	F	F
F	V	V
F	F	V

Logique propositionnelle

Sémantique

Équivalence ($P \leftrightarrow Q$)

P	Q	$P \leftrightarrow Q$
V	V	V
V	F	F
F	V	F
F	F	V

Logique des propositions

.
À l'aide des **tables de vérité**, on peut facilement déterminer la **véracité** d'un énoncé dans un modèle donné, grâce à une simple évaluation récursive

Exemple: à l'aide d'une table de vérité déterminez la véracité de l'énoncé

$$\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1})$$

dans le modèle $m_1 = \{P_{1,2} = \text{faux}, P_{2,2} = \text{faux}, P_{3,1} = \text{vrai}\}$

$$\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1}) = \text{Vrai} \wedge (\text{Faux} \vee \text{Vrai}) = \text{Vrai} \wedge \text{Vrai} = \text{Vrai}$$

Limites de la logique des propositions

- ❑ La logique propositionnelle ne permet de décrire que des constructions simples du langage, consistant essentiellement en des opérations **booléennes** sur les propositions.
- ❑ On peut, grâce à elle, étudier dans un cadre formel la valeur de vérité de formules relativement peu expressives.
- ❑ Ne permet pas non plus de décrire des relations entre plusieurs variables.
- ❑ Insuffisante pour représenter des procédés de langage effectivement utilisés en informatique, linguistique ou en mathématiques.

Limites de la logique des propositions

Expressivité limitée

La logique des propositions manipule uniquement des propositions atomiques (comme P, Q) et des connecteurs logiques : ($\neg, \vee, \wedge, \rightarrow, \leftrightarrow$),

- Elle **ne peut pas exprimer** :
- des **relations internes** à une proposition,
- des **structures complexes** comme « *tout étudiant aime les mathématiques* » ou « *il existe un nombre pair qui est premier* ».

Pas de quantificateurs

Impossible d'utiliser les quantificateurs comme :

$\forall$ (pour tout)

$\exists$ (il existe)

Exemple non exprimable en logique propositionnelle :

$\forall x (\text{Étudiant}(x) \Rightarrow \text{AimeMaths}(x))$

Limites de la logique des propositions

Pas de variables ni de fonctions

La logique des propositions ne permet pas de manipuler des variables individuelles (comme x , y) ni des fonctions (comme $f(x)$).

Elle ne peut donc pas modéliser des relations comme :

« x est l'ami de y »

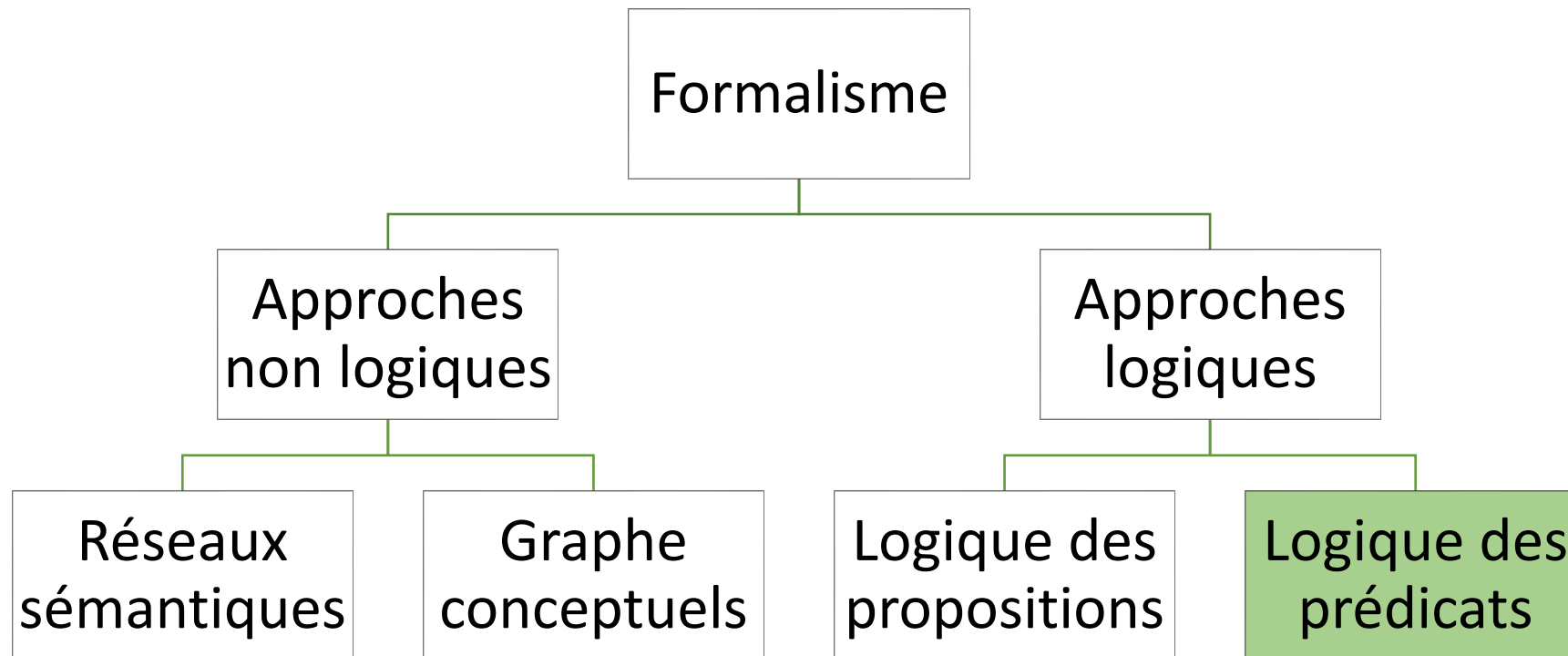
« $f(x) = x^2$ »

Utilité réduite en informatique avancée

➤ Insuffisante pour :

- La **programmation logique** (comme Prolog)
- Le **raisonnement en IA**
- La **représentation formelle des mathématiques**
- Dans ces domaines, on utilise des formalismes plus puissants comme la **logique des prédicats** (ou logique du premier ordre).

Formalismes de représentation de connaissance



Logique des prédicats

- ❑ Les éléments de base du langage ne sont plus des propositions mais des prédicats.
- ❑ Un prédicat **P** est une fonction d'un nombre fini de variables ou de constantes.
- ❑ Un prédicat peut être vu comme une fonction propositionnelle
La mer est bleue , le prédicat est **bleu (x)**
- ❑ Le prédicat **aimer** prend deux arguments : l'être aimant et l'être ou la chose aimée. **aime(x,y)**

Logique des prédicats

Le vocabulaire de la logique des prédicats est constitué de cinq classes de symboles :

- Les constantes (a,b,c,d ...)
- Les variables (w,x,y ...)
- Les symboles de prédicats positive ou nulle (A,B ...)
- Les connecteurs logiques
- Les quantificateurs ($\forall$, $\exists$)

Logique des prédicats

- Jean est plus grand que Marie

$$G(j, m)$$

- Paul a vu Léa et elle ne l'a pas vu

$$V(p, l) \wedge \neg V(l, p)$$

- Si Jean est un homme, alors il est mortel

$$H(j) \rightarrow M(j)$$

- Un chat est entré

$$\exists x(C(x) \wedge E(x))$$

- Certains enfants ne sont pas malades

$$\exists x(E(x) \wedge \neg M(x))$$

- Tous les éléphants ont une trompe

$$\forall x(E(x) \rightarrow T(x))$$

- Tous les hommes n'aiment pas Marie

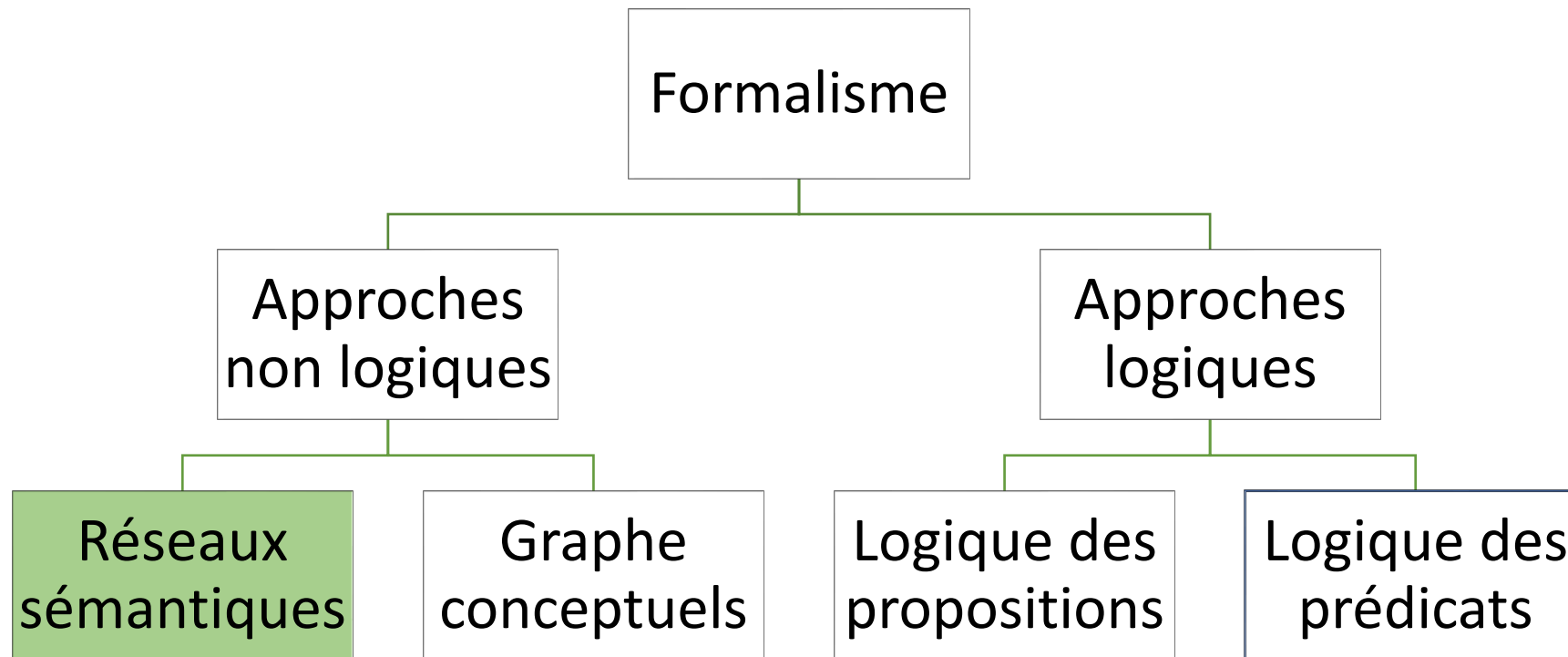
$$\forall x(H(x) \rightarrow \neg A(x, m))$$

Approches non logiques

- ❑ Réseaux sémantiques

- ❑ Graphes conceptuels

Formalismes de représentation de connaissance



Réseaux sémantiques

- Un réseau sémantique est une structure de données utilisée pour représenter des connaissances sous forme de graphes.
- Les nœuds représentent des concepts ou objets, les arcs représentent des relations sémantiques (ex : "est un", "partie de", "possède").

Réseaux sémantiques

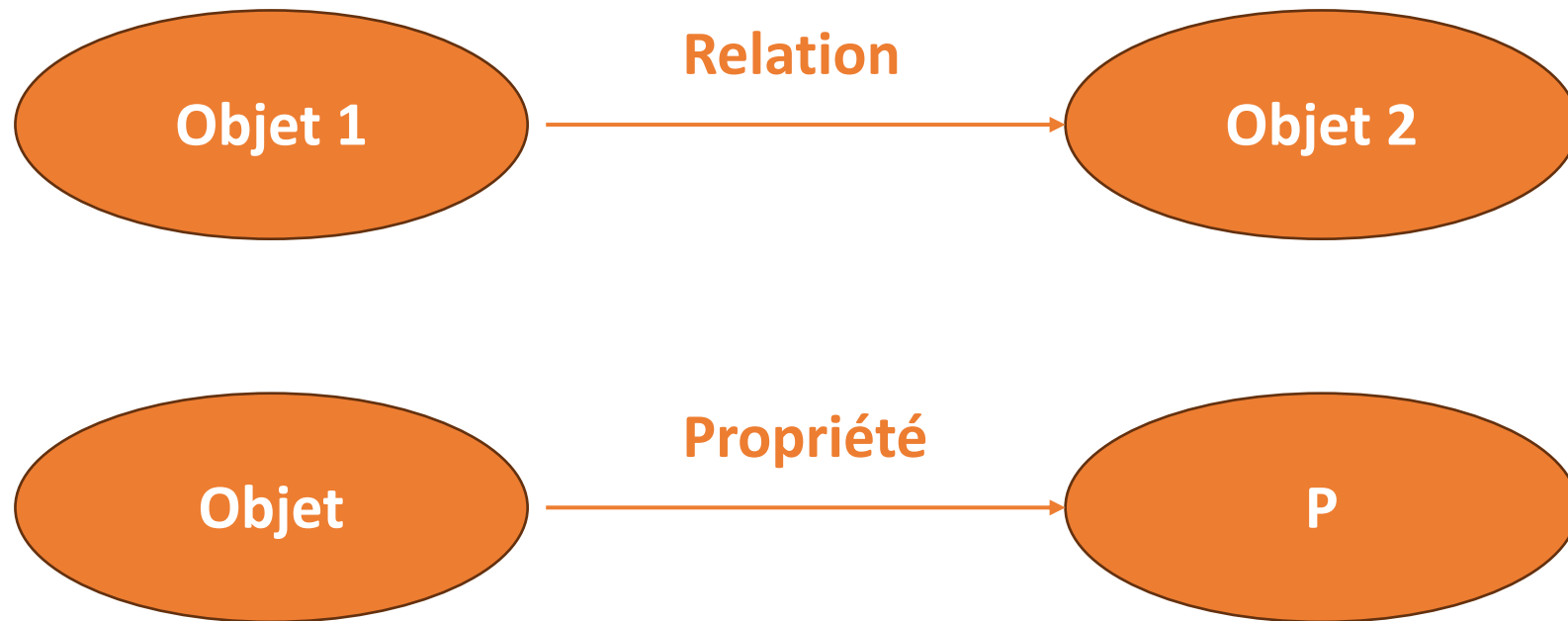
Arcs

- Typiquement :
 - est_un (is-a) : relation d'héritage
 - a_pour_partie : relation de composition
 - a_pour_propriété: Possession
- ❑ **est_un** : relation entre un individu et une classe exprimant l'appartenance.
- ❑ **sorte_de** : relation entre deux classes exprimant l'inclusion.

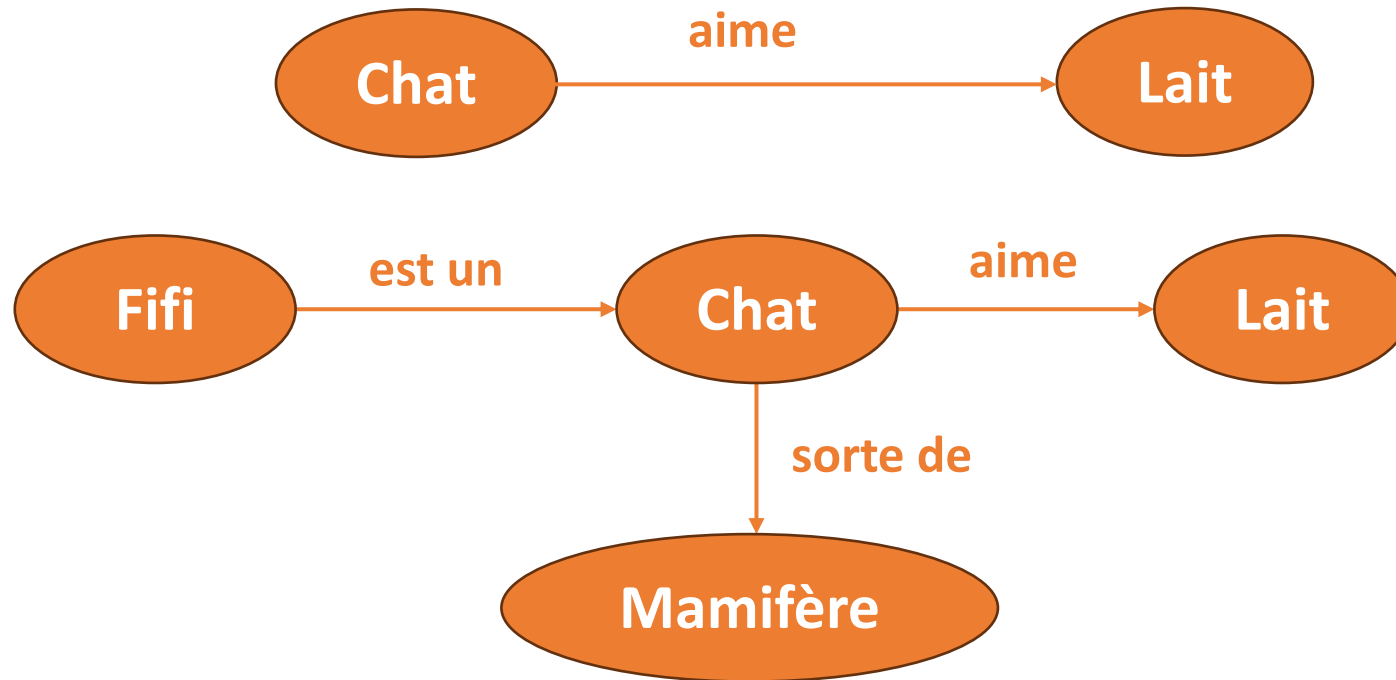
Réseaux sémantiques

- Représentation graphique directement interprétable par l'humain
 - Idée de base :
 - modéliser le mécanisme d'association d'idées
 - Facilité de traduction du langage naturel vers ce formalisme
 - utiliser les classifications
- Formalisme directement exploitable par une machine

Réseaux sémantiques



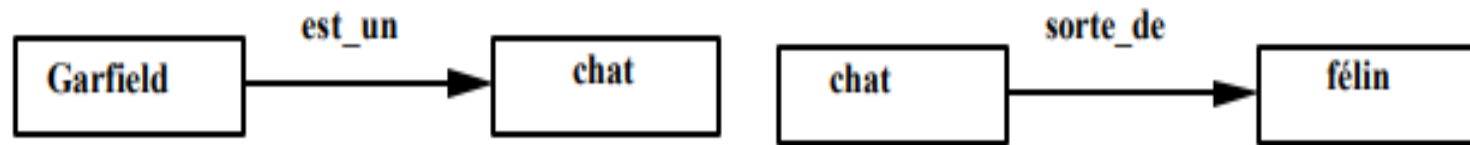
Réseaux sémantiques



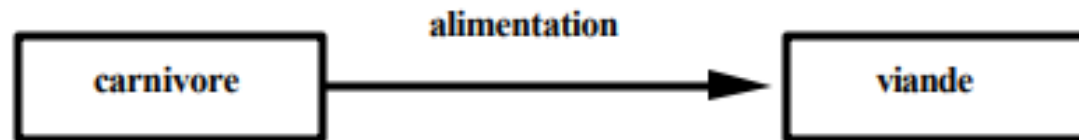
Réseaux sémantiques

❑ Garfield est un chat

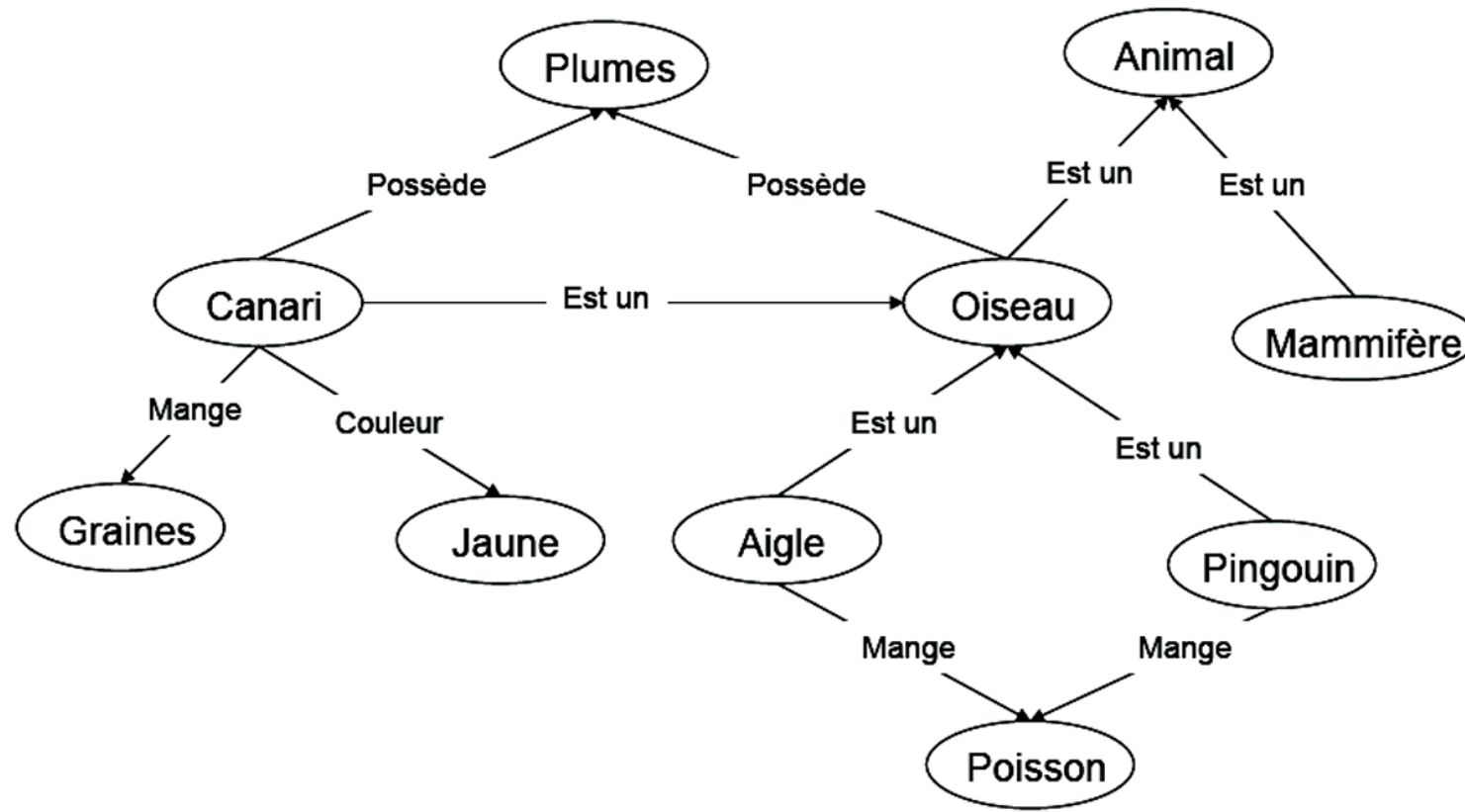
❑ Les chats sont des félins



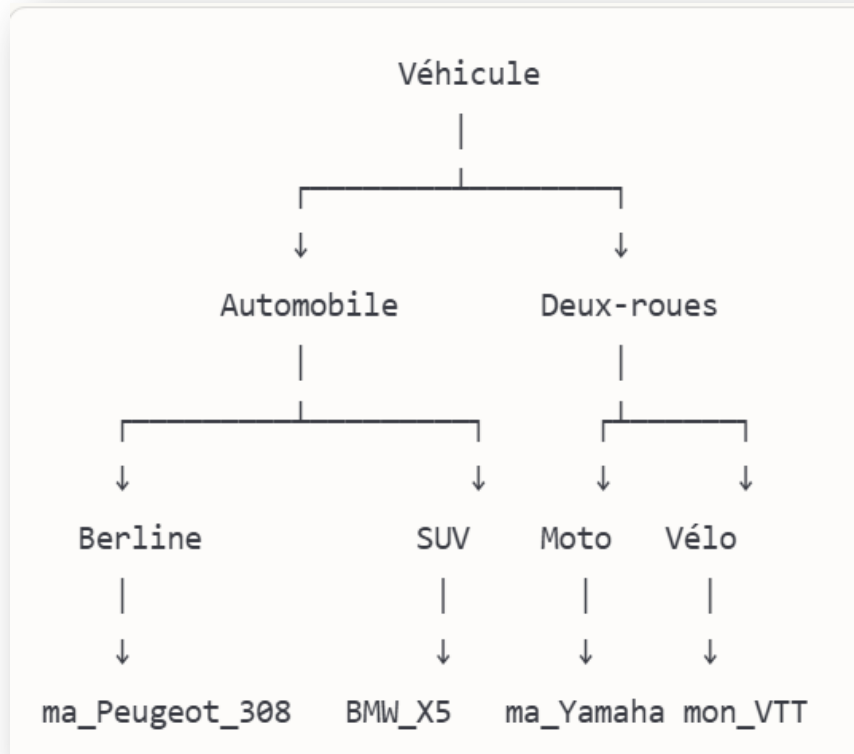
❑ Les carnivores mangent de la viande



Réseaux sémantiques



Réseaux sémantiques: exemples



Relations "sorte_de" (entre classes) :

Automobile --sorte_de--> Véhicule

Deux-roues --sorte_de--> Véhicule

Berline --sorte_de--> Automobile

SUV --sorte_de--> Automobile

Moto --sorte_de--> Deux-roues

Vélo --sorte_de--> Deux-roues

Relations "est_un" (instance vers classe) :

ma_Peugeot_308 --est_un--> Berline

BMW_X5 --est_un--> SUV

ma_Yamaha --est_un--> Moto

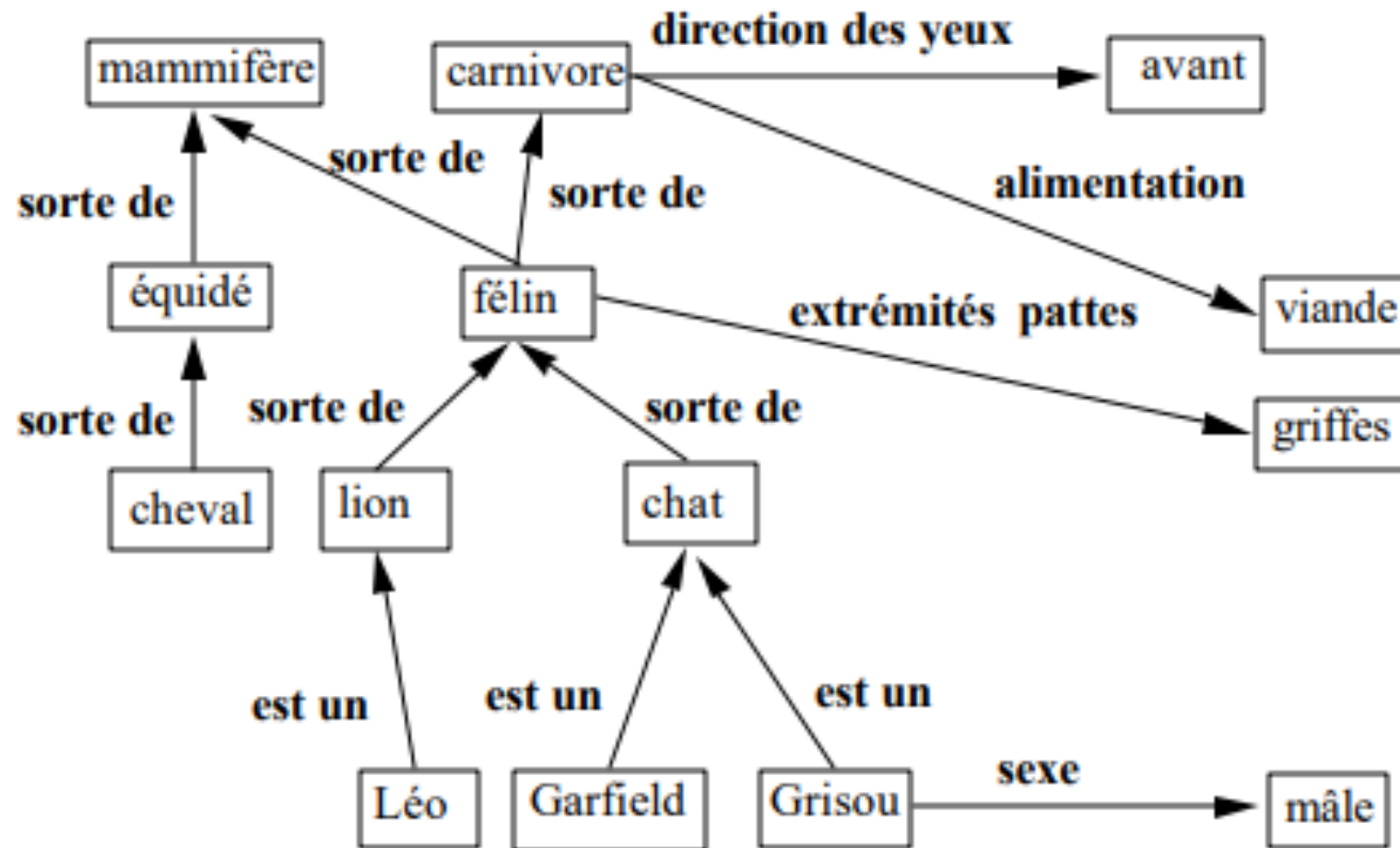
mon_VTT --est_un--> Vélo

Réseaux sémantiques: exemples

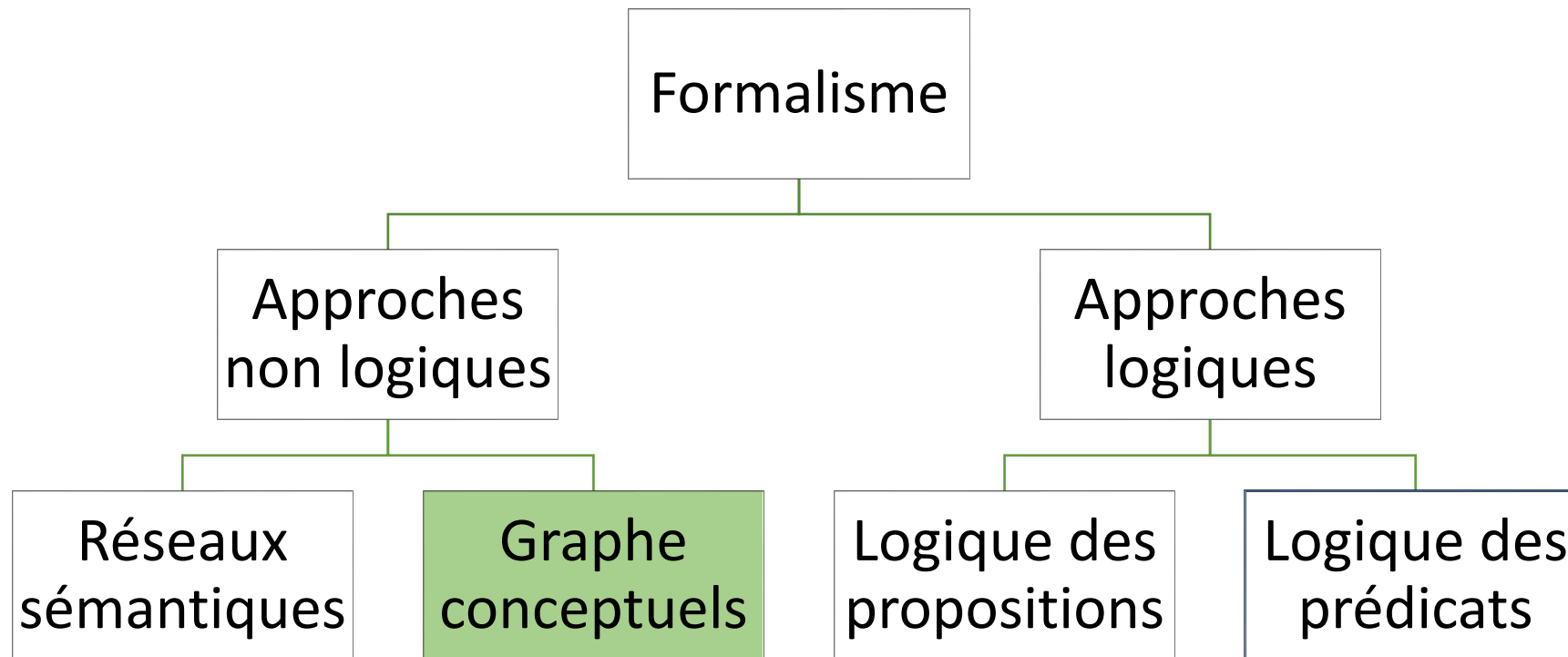
□ Un félin est un carnivore. Un carnivore est un animal qui a les yeux dirigés vers l'avant et qui mange de la viande. Les pattes d'un félin ont des griffes à leurs extrémités. Un félin est un mammifère. Grisou et Garfield sont des chats. Grisou est un chat mâle. Léo est un lion. Les chats et les lions sont des félins. Un cheval est un équidé. Un équidé est un mammifère.

- félin **sorte_de** carnivore
- carnivore **direction des yeux** avant
- carnivore **alimentation** viande
- félin **extrémités pattes** griffes
- félin **sorte_de** mammifère
- Grisou **est_un** chat
- Garfield **est_un** chat
- Grisou **sexe** mâle
- Léo **est_un** lion
- chat **sorte_de** félin
- lion **sorte_de** félin
- cheval **sorte_de** équidé
- équidé **sorte_de** mammifère

Réseaux sémantiques: exemples



Formalismes de représentation de connaissance



Graphes conceptuels

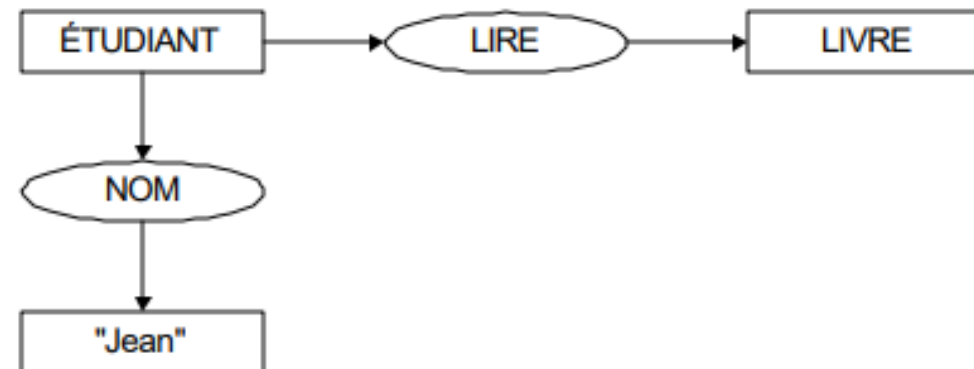
- ❑ Les graphes conceptuels sont des graphes **finis**, **connexes** et **bipartites**.
- ❑ Les deux sortes de nœuds d'un graphe bipartite sont les **concepts** et les **relations conceptuelles**.
- ❑ Un concept est un couple : (type, référent)
- ❑ Chaque relation conceptuelle possède un **arc** ou plus, chacun d'entre eux devant être relié à un concept.

Graphes conceptuels

□ Notation graphique

- Concepts : rectangles
- Relations : ovales
- Arcs : flèches

Exemple: Jean lit un livre



Programmation logique

Programmation logique

- ❑ La programmation logique est un type de paradigme de programmation qui est en grande partie basée sur la logique formelle.
- ❑ Tout programme écrit dans un langage de programmation logique est un ensemble de phrases sous forme logique, exprimant des faits et des règles concernant un domaine problématique.
- ❑ Ces faits et ces règles sont exploités par un moteur d'inférence, en réaction à une question ou requête.

Programmation logique

- ❑ La programmation logique est une variété de ce que nous connaissons avec la programmation déclarative, un paradigme **opposé** à pratiquement tous les langages de programmation populaires tel que l'assembleur, C, Java ou Python qui font partie de la programmation impérative.
- ❑ La programmation logique est considérée comme une programmation déclarative plutôt qu'impérative, car elle s'attache davantage au *quoi* qu'au *comment*, le moteur assumant une large part des enchaînements.
- ❑ Particulièrement adaptée aux besoins de l'intelligence artificielle, dont elle est un des principaux outils.

Programmation impérative vs logique

- ❑ Ce paradigme **est basé sur la formule “algorithmes = logique + contrôle”** (l'équation informelle de Kowalski), ce qui signifie qu'un algorithme est créé en spécifiant des connaissances par des axiomes (logique) et que le problème est résolu par un mécanisme d'inférence qui agit sur lui (contrôle).
- ❑ Parmi les langages de programmation logique **on peut mettre en évidence Prolog, Lisp ou Erlang.**

PROLOG

- ❑ Dérivé de **PRO**grammation en **LOG**ique
- ❑ Né en 1972
- ❑ Langage de programmation déclaratif basé sur la logique du premier ordre
- ❑ Basé sur la logique des prédicats
- ❑ Un programme Prolog se compose d'un **ensemble de faits** (affirmations simples) et règles (qui servent à affirmer la véracité d'un fait basé sur d'autres).
- ❑ L'ensemble des faits d'un programme équivaut à avoir une base de données qu'on appelle « **base de connaissances** ».
- ❑ Le programme Prolog = **base de faits + règles + requêtes**
- ❑ Utilisé dans l'Intelligence Artificielle, les systèmes experts, le traitement du langage naturel.

PROLOG

Éléments de base de Prolog

1. Faits (fact)

Ce sont des affirmations simples considérées comme vraies.

`homme(pierre).`

`femme(marie).`

`parent(pierre, marie)`

Ici :

`homme(pierre)` signifie : Pierre est un homme.

`parent(pierre, marie)` signifie : Pierre est le parent de Marie.

PROLOG

Éléments de base de Prolog

2. Règles (rule)

Ce sont des implications : elles permettent de déduire de nouveaux faits.

pere(X, Y) :- homme(X), parent(X, Y).

Cela se lit : « X est le père de Y si X est un homme et X est le parent de Y. »

3. Requêtes (questions) (query)

Utilisées pour interroger la base de connaissances.

?- **pere(pierre, marie).**

Exemple simple complet

homme(pierre).

femme(marie).

parent(pierre, marie).

pere(X, Y) :- homme(X), parent(X, Y).

?- **pere(pierre, marie).**

PROLOG

Pourquoi utiliser Prolog ?

Domaine	Utilisation
Intelligence artificielle	Systèmes experts, moteurs d'inférence
Linguistique	Analyse syntaxique, traitement du langage naturel
Apprentissage automatique	Modélisation logique de connaissances et d'hypothèses
Informatique théorique	Représentation formelle de raisonnements logiques

PROLOG

Pourquoi utiliser Prolog ?

Système expert : un système expert est un programme informatique conçu pour simuler le raisonnement d'un expert humain dans un domaine spécifique.

Composants principaux :

Élément	Rôle
Base de connaissances	Contient des faits et des règles (ex : "Si le patient a de la fièvre et touse, alors il est peut-être malade").
Moteur d'inférence	Cerveau du système : interprète les règles pour déduire des conclusions à partir des faits donnés.
Interface utilisateur	Permet à l'utilisateur de fournir des données et de recevoir des réponses .

PROLOG

Pourquoi utiliser Prolog ?

Exemple :

Un système expert médical pourrait :

Recevoir des symptômes comme données,

Utiliser ses règles pour poser un **diagnostic**.

2. Moteur d'inférence :

Le moteur d'inférence est le composant logique d'un système expert. 🔍

Fonctionnement :

Il applique un raisonnement automatique basé sur :

- Des faits connus (ex : "le patient a 39°C de fièvre")
- Des règles logiques (ex : "si fièvre > 38°C alors état fiévreux")

Il enchaîne les règles pour déduire de nouvelles informations, comme le ferait un humain.

PROLOG

Pourquoi utiliser Prolog ?

fièvre(pierre).

tousse(pierre).

malade(X) :- fièvre(X), tousser(X).

Pratique - faits

- Faits = affirmations simple

```
humain(socrate).
```

```
chat(tom).
```

- Chaque fait termine par un **point**.

Pratique - Règles

- Règle = condition logique

```
mortel(X) :- humain(X).
```

- Se lit : “X est mortel si X est humain”.

Partique – *Requêtes*

- Poser une question à Prolog

```
?- mortel(socrate). % Résultat : true
```

- ?- indique une requête à exécuter.

Prolog

Éléments de base en Prolog

Élément	Exemple	Description
Constante	socrate	Objet spécifique
Variable	X	Commence par une majuscule
Prédicat	homme(X)	Relation logique
Liste	[a, b, c]	Structure de données
Commentaires	% ou /* ... */	Pour commenter le code

Prolog

Exemple de raisonnement

```
homme(socrate).  
mortel(X) :- homme(X).  
?- mortel(socrate). % Résultat : true
```

- Prolog cherche une **preuve logique** à la requête.

Prolog

Contrôle logique

- Conjonction (et) : ,
- Disjonction (ou) : ;
- Négation : \+

Prolog

Mini projet – Arbre généalogique

```
parent(paul, pierre).
```

```
parent(marie, pierre).
```

```
homme(paul).
```

```
pere(X,Y) :- parent(X,Y), homme(X).
```

- Questions possibles : ?- pere(paul, pierre).

Prolog

Environnement Prolog

Console

<https://www.swi-prolog.org/download/stable/bin/swipl-9.2.9-1.x64.exe>

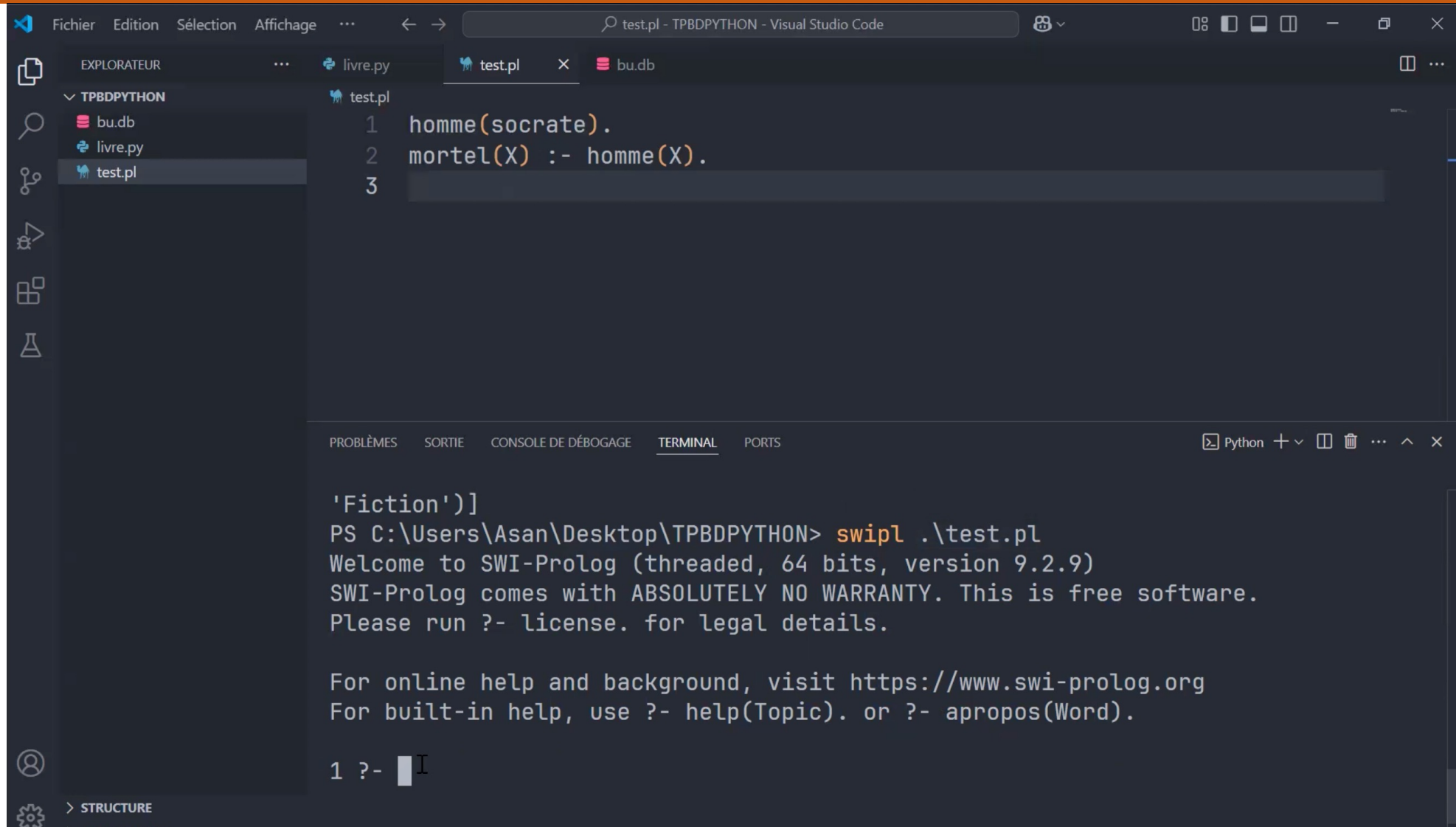
NB: Accès commande console depuis vscode: `swipl test.pl`
Et on tape après la requête.

editor

<https://arbeitsplattform.bildung.hessen.de/fach/informatik/swiprolog/swiprologeditorsetup64.exe>

Prolog

Environnement Prolog



The screenshot displays the Visual Studio Code interface with a workspace named 'TPBDPYTHON'. The Explorer sidebar on the left shows the file structure with 'test.pl' selected. The main editor window shows the contents of 'test.pl' with the following Prolog code:

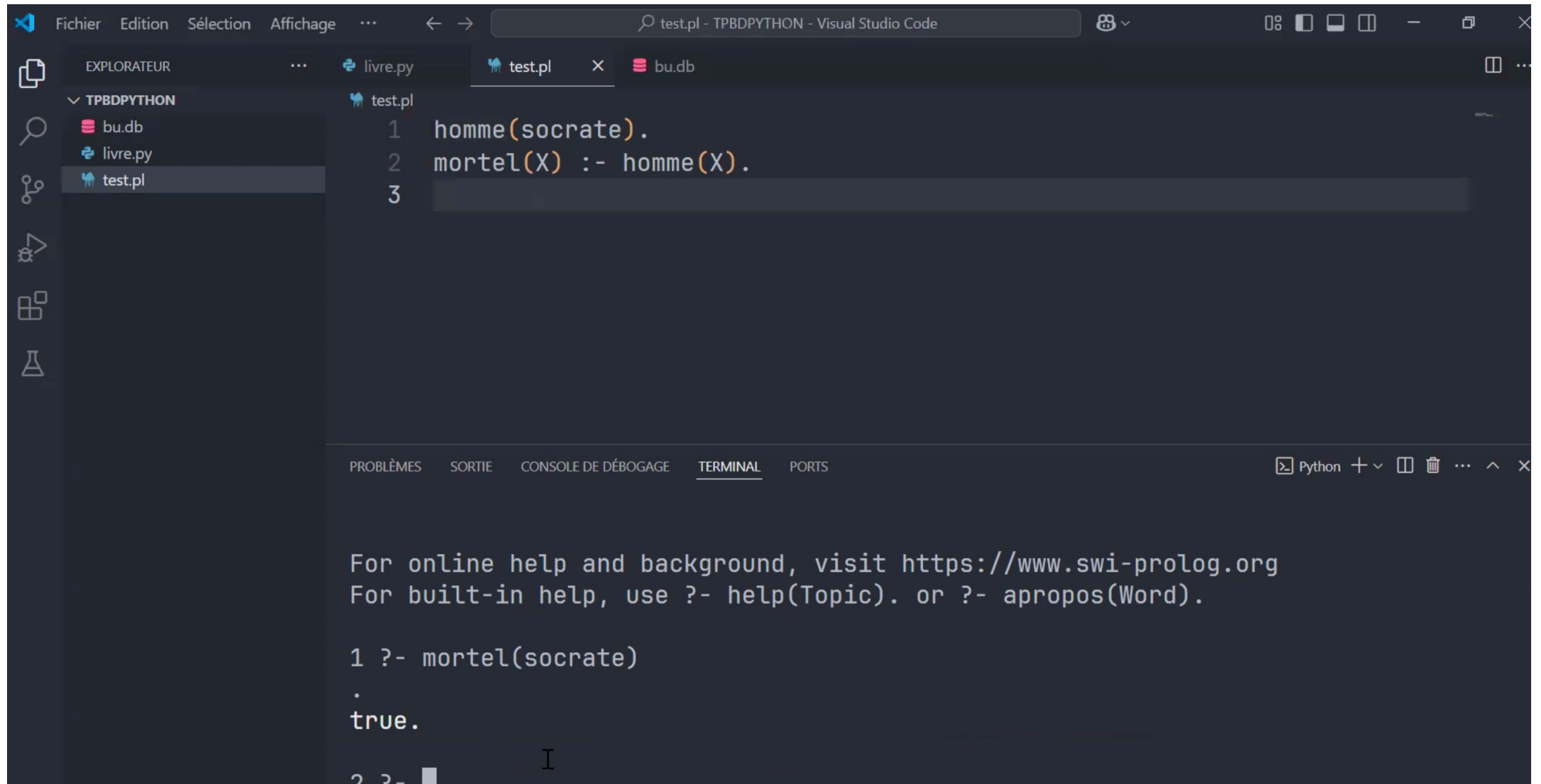
```
1 homme(socrate).  
2 mortel(X) :- homme(X).  
3
```

The bottom panel shows the 'TERMINAL' output, which includes the command 'swipl .\test.pl' and the resulting output from the SWI-Prolog interpreter:

```
'Fiction']]  
PS C:\Users\Asan\Desktop\TPBDPYTHON> swipl .\test.pl  
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.9)  
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.  
Please run ?- license. for legal details.  
  
For online help and background, visit https://www.swi-prolog.org  
For built-in help, use ?- help(Topic). or ?- apropos(Word).  
  
1 ?- █
```

Prolog

Environnement Prolog



```
test.pl - TPBDPYTHON - Visual Studio Code
```

EXPLORATEUR

- TPBDPYTHON
 - bu.db
 - livre.py
 - test.pl

```
1 homme(socrate).
2 mortel(X) :- homme(X).
```

PROBLÈMES SORTIE CONSOLE DE DÉBOGAGE TERMINAL PORTS

Python + -

For online help and background, visit <https://www.swi-prolog.org>
For built-in help, use ?- help(Topic). or ?- apropos(Word).

```
1 ?- mortel(socrate)
.
true.

2 ?-
```

FIN Chapitre 2

Chapitre 3:

Systèmes experts

Sommaire

- Définition
- Historique
- Structure
- Fonctionnement
 - Chainage avant
 - Chainage arrière
 - Comparaison
- Exemple

Définition d'un Système Expert (SE)

- Programme qui permet l'exploitation des connaissances dans un domaine précis et rigoureusement limité.
- Tel un expert humain, un SE n'a aucune prétention en dehors de sa spécialité.
- Un système informatique où les données (la base de connaissance) sont bien séparées du programme qui les manipule (le moteur d'inférences).
- La base de données des connaissances ainsi que le système d'inférence sont fondés sur la logique et l'inférence logique.
- Outil capable de reproduire les mécanismes cognitifs d'un expert, dans un domaine particulier.
- Logiciel capable de répondre à des questions, en effectuant un raisonnement à partir de faits et de règles connus. Il peut servir notamment comme outil d'aide à la décision.
- **Un SE est un système informatique qui utilise les connaissances d'un expert pour obtenir une performance de décision élevée dans un domaine spécifique.**

HISTORIQUE

Dendral

- Premier système expert
- Créé en 1965
- Créé par les informaticiens Edward Feigenbaum, Bruce Buchanan, le médecin Joshua Lederberg et le chimiste Carl Djerassi.
- ✓ Permettait d'identifier les constituants chimiques d'un matériau à partir de spectrométrie de masse et de résonance magnétique nucléaire.

HISTORIQUE

Mycin

- Le plus connu, créé en 1972
- Système expert de diagnostic de maladies du sang et de prescription de médicaments, avec un vrai moteur et une vraie base de règles.
- Les règles étaient affectées de coefficients de vraisemblance qui donnaient à chacune d'entre elles un poids particulier face aux autres.

Pourquoi les SE

- Il y a plusieurs raisons pour lesquelles une entreprise a besoin d'un système expert:
- Des experts dans l'entreprise peuvent se retirer ou partir.
- Un système expert permet de conserver des connaissances. Ils permettent de documenter des connaissances.
- Ils permettent de former et d'éduquer des nouveaux employés. Ils permettent un transfert rapide et peu coûteux des connaissances.

Qu'est-ce qu'un expert?

- Un expert est une personne qui a des connaissances, jugements, expériences et méthodes sur un domaine précis (un expert dans une entreprise n'est pas nécessairement un expert dans une autre entreprise) et capable de les exploiter pour donner des conseils et résoudre des problèmes.
- L'expert est responsable de donner les connaissances nécessaires pour que le système expert fasse sa tâche car il sait quelle information est importante et comprend les relations entre les différents faits.

Les SE

Un expert est capable de:

- Reconnaître et formuler le problème à résoudre.
- Résoudre le problème rapidement et correctement.
- Expliquer la solution.
- Apprendre avec l'expérience.
- Restructurer des connaissances.
- Briser les règles si nécessaire.
- Déterminer la pertinence.
- Être conscient de ses limites.

Les Domaines d'applications d'un SE

Les SE se retrouvent utilisés dans divers domaines d'applications tels que:

- **La médecine** : diagnostic médical, classification radiographique.
- **L'informatique** : composante de systèmes tutoriels intelligents, maintenance électronique.
- **Le marketing**: sélection d'hôtel.
- **L'industrie** : surveillance, diagnostics de pannes, gestion d'incidents

Qu'est-ce qu'une expertise?

- C'est la connaissance spécifique à une tâche que connaît l'expert.
- Le niveau d'expertise détermine la performance d'une décision.
- L'expertise est souvent obtenue par la formation, la lecture et l'expérience.

Classification des SE

3 classes:

➤ **ordre 0**

- fondé sur le calcul propositionnel.
- Les faits sont à valeurs booléennes.

Exemple : SI `agé_de_plus_de_18_ans` ALORS majeur

Classification des SE

➤ ordre 0+ ou 0.5

- les faits peuvent être à valeurs réelles ou symboliques

Exemple : SI $\text{age} \geq 18$ ALORS $\text{statut} = \text{majeur}$

➤ ordre 1

- basés sur le calcul des prédicats
- utilisation des variables et de l'unification

Exemple : SI $\text{age}(X, \text{Age})$ ET $\text{Age} \geq 18$ ALORS $\text{majeur}(X)$

Structure d'un SE

Les systèmes experts sont généralement constitués de :

- Une base de connaissance (*knowledge base*)
- Une interface
- Un moteur d'inférence (*Inference Engine*)

La base de connaissances (BC)

- Ensemble des données qui sont utilisées par le moteur d'inférence.
- C'est là qu'est stocké le savoir du système propre au domaine de connaissance.
- Elle rassemble toutes les connaissances d'un expert du domaine considéré.

La base de connaissance (BC)

- **Les standards d'engagement** (connaissances de l'expert) : l'informations de base et de configuration du système, mesures, lois, paramètres, données contractuelles.
- **Les règles d'inférence** (savoir-faire) : ensemble des règles logiques de déduction utilisées par le moteur d'inférence.
- **La base de faits** (expérience) Historisation et statistique des faits effectifs, des décisions et des buts. Le raisonnement va se baser sur ces faits pour déduire des conclusions.

La base de faits

- Contient l'ensemble des données de départ sur lequel le système va commencer à travailler.
- Cette base s'enrichit au fur et à mesure des déductions faites par le système.
- Cet espace de travail est un peu la mémoire à court terme, le système y stocke aussi les règles en attente, les sous-problèmes ...

Les interfaces

- Dialogue entre l'expert, chargé de la création de la base de connaissances et la machine.
- Dialogue entre l'utilisateur et le système pour l'entrée des faits de la base de faits à exploiter.

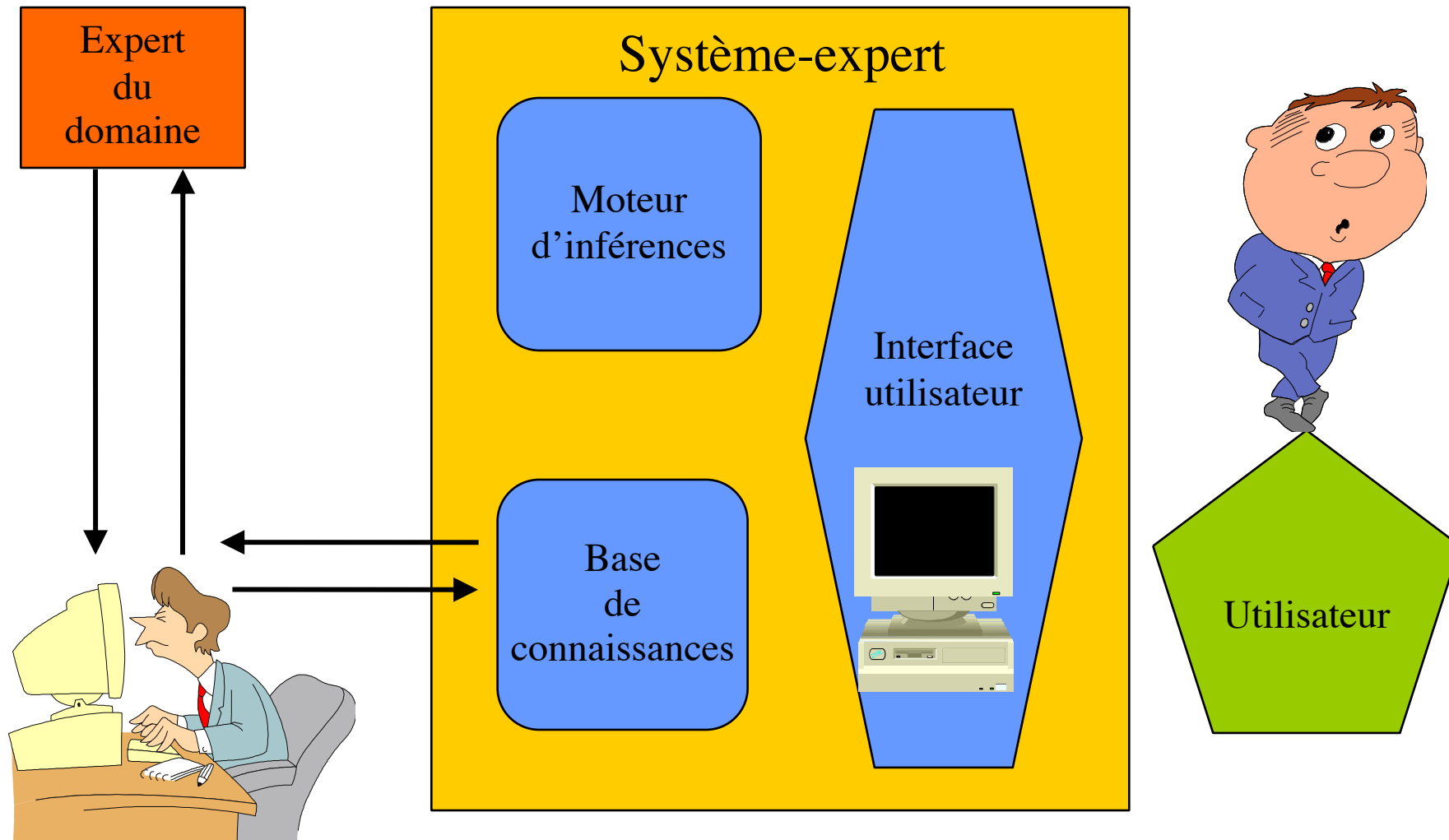
Le moteur d'inférence

- Mécanisme qui permet d'inférer des connaissances nouvelles à partir de la base de connaissances du système.
- Cerveau du système
- Sert à déclencher les règles et à les enchaîner les unes après les autres.
- Les deux mécanismes les plus employés pour déclencher les règles sont :
 - Le chaînage avant
 - Le chaînage arrière

Le moteur d'inférence

- Il est important que la base de connaissance reste indépendante du moteur d'inférence (sauf si elle contient les règles d'inférence elles-mêmes).
- Ce moteur d'inférence est écrit dans un langage d'intelligence artificielle, tels que LISP ou PROLOG.

Fonctionnement d'un SE



MISE EN PLACE

1. Etude de faisabilité

Evaluer le domaine et les risques d'échecs de la mise en place et de succès de l'outil auprès des professionnels et futurs utilisateurs.

2. Extraction des données

Dialogue entre le cognitif et l'expert afin d'extraire de ce dernier toutes ses connaissances et son savoir-faire.

3. Formalisation

Formaliser les connaissances qu'il a récolté.

4. Design et développement

Faire appel aux développeurs informatique afin de commencer à définir le cahier des charges précis, la base de connaissance et les règles d'inférence.

5. Tests et optimisations

Une série de tests auprès des experts mais aussi auprès d'utilisateurs lambdas qui sont sensés à partir de cet outil fournir les résultats d'un expert débutant.

Acquisition des données

- Une tâche longue et fastidieuse
- Les algorithmes de manipulation de faits et de règles sont nombreux et connus.
- La détermination de l'ensemble des faits et règles qui vont composer la base de connaissances est un problème délicat.
- Il faut décrire le comportement d'un expert face à un problème particulier, et sa manière de le résoudre. Car ce que l'on souhaite obtenir n'est ni plus ni moins que l'expérience, la connaissance pratique de l'expert, et non la théorie que l'on peut trouver dans les livres.

Résumé

- **Faits:** Ensemble de vérités connues
- **Règles :**
 - Partie condition : prémisses
 - Partie action : conclusion, calcul, affichage ...
 - Éventuellement : coefficient de confiance

SI **condition** ALORS **action** [coefficient]

Fonctionnement

Chaînage avant (Raisonnement guidé par les données)

- Détecter les règles dont les prémisses sont vérifiées
- Sélectionner la règle à appliquer
- Appliquer la règle
- Recommencer jusqu'à ce qu'il n'y ait plus de règle applicable

Chaînage arrière (Raisonnement guidé par le but)

- Détection des règles qui concluent à ce but
- Résolution de conflits
- Application de la règles (les éléments des prémisses deviennent de nouveau sous-buts à atteindre).
- Arrêt : pile vide ou aucune règle applicable

Chainage avant et chainage arrière

- **Chainage avant**

Se base sur le Modus Ponens

si $f1$ est vrai et $f1 \Rightarrow f2$ alors $f2$ est vrai.

- **Chainage arrière**

Se base sur le Modus Tollens

Si $f2$ est non vrai et si $f1 \Rightarrow f2$ alors $f1$ est non vrai.

Chainage avant VS arrière

- Capacité d'explication et la transparence est plus développée en chaînage arrière qu'en chaînage avant parce que toutes les questions sont posées pour répondre à des sous buts.
- En chaînage avant les questions sont posées selon l'ordre de présentation des règles dans la base de connaissances et peuvent parfois refléter une incohérence.
- Le système peut poser des questions inutiles en chaînage avant, ce qui n'est pas le cas en chaînage arrière.

Exemple

- On considère le système expert « **pâtisserie** »

Base de règles	Base de faits
R1 : Si farine et beurre et œufs et sel alors pâte R2 : Si pommes et sucre alors pommes sucrées R3 : Si pommes sucrées et pâte alors tarte aux pommes R4 : Si abricots et pâte alors tarte aux abricots R5 : Si poires et pâte alors tarte aux poires R6 : Si cerises et pâte alors tarte aux cerises	Pommes Poires Abricots Farine Beurre œufs Sucre Sel

Résolution

- **Chainage avant** : Avec les ingrédients (les faits) qu'est-ce que je peux faire ?
- **Chainage arrière** : Est-ce que je peux faire une tarte aux abricots ?

Chainage avant

- **R1 : est-elle applicable ?**

Oui (tous les faits sont dans la base des faits)

Le fait **pâte** est donc rajouté dans la base des faits et la R1 est désactivée.

Base de règles	Base de faits
R1 : Si farine et beurre et œufs et sel alors pâte R2 : Si pommes et sucre alors pommes sucrées R3 : Si pommes sucrées et pâte alors tarte aux pommes R4 : Si abricots et pâte alors tarte aux abricots R5 : Si poires et pâte alors tarte aux poires R6 : Si cerises et pâte alors tarte aux cerises	Pommes Poires Abricots Farine œufs Beurre Sucre Sel Pâte

Chainage avant

- **R2 : est-elle applicable ?**

Oui

Le fait pommes sucrées est ajouté dans la base des faits et la R2 est désactivée.

Base de règles	Base de faits
R1 : Si farine et beurre et œufs et sel alors pâte R2 : Si pommes et sucre alors pommes sucrées R3 : Si pommes sucrées et pâte alors tarte aux pommes R4 : Si abricots et pâte alors tarte aux abricots R5 : Si poires et pâte alors tarte aux poires R6 : Si cerises et pâte alors tarte aux cerises	Pommes Poires Abricots Farine Beurre Œufs Sucre Sel Pâte Pommes sucrées

Chainage avant

- De même, les règles R3, R4 et R5 sont sélectionnées et exécutées.

Base de règles	Base de faits
R1 : Si farine et beurre et œufs et sel alors pâte R2 : Si pommes et sucre alors pommes sucrées R3 : Si pommes sucrées et pâte alors tarte aux pommes R4 : Si abricots et pâte alors tarte aux abricots R5 : Si poires et pâte alors tarte aux poires R6 : Si cerises et pâte alors tarte aux cerises	Pommes Poires Abricots Farine Beurre Œufs Sucre Sel Pâte Pommes sucrées Tarte aux pommes Tarte aux abricots Tarte aux poires

Chainage avant

- **Base de faits initiale** : Pommes, Poires, Abricots, Farine, Beurre, Œufs, Sucre, Sel
- **Base de règles** :
 - **R1** : Si farine et beurre et œufs et sel → pâte
 - **R2** : Si pommes et sucre → pommes sucrées
 - **R3** : Si pommes sucrées et pâte → tarte aux pommes
 - **R4** : Si abricots et pâte → tarte aux abricots
 - **R5** : Si poires et pâte → tarte aux poires
 - **R6** : Si cerises et pâte → tarte aux cerises

Chainage avant

- **Raisonnement par chaînage avant**

Objectif : Avec les faits actuels, que peut-on produire ?

- **Étape 1 :**

On regarde si une règle est applicable avec les faits existants.

✓ **R1** est applicable car les faits contiennent : farine, beurre, œufs, sel

Donc on ajoute "pâte" aux faits.

✗ R1 est ensuite désactivée (déjà utilisée).

- **Nouvelle base de faits :**

Pommes, Poires, Abricots, Farine, Beurre, Œufs, Sucre, Sel, **Pâte**

Chainage avant

- **Raisonnement par chaînage avant**
- **Étape 2 :**
 - ✓ **R2** est applicable : pommes + sucre → pommes sucrées
On ajoute **pommes sucrées**
 - ✗ R2 désactivée
- **Nouvelle base de faits :**
- ... + **pommes sucrées**
- **Étape 3 :**
 - ✓ **R3** : pommes sucrées + pâte → tarte aux pommes
On ajoute **tarte aux pommes**

Chainage avant

- **Raisonnement par chaînage avant**
- **Étape 4 :**
 - ✓ **R4** : abricots + pâte → tarte aux abricots
On ajoute **tarte aux abricots**
- **Étape 5 :**
 - ✓ **R5** : poires + pâte → tarte aux poires
On ajoute **tarte aux poires**
- **Étape 6 :**
 - ❖ **R6** : cerises + pâte → tarte aux cerises
✗ cerises n'est **pas** dans les faits → R6 **non applicable**

Chainage avant

Résultat final (chaînage avant)

À partir des faits initiaux, on peut faire :

- pâte
- pommes sucrées
- tarte aux pommes
- tarte aux abricots
- tarte aux poires

Chainage avant

- **R6 : elle ne peut pas être exécutée.**
Le fait "cerises" n'existe pas dans la base des faits.
- **Le mécanisme ne peut plus effectuer de déductions et le moteur s'arrête.**

Chainage arrière

- But : Tarte aux abricots.
- Considérons R4 : les prémisses sont-elles vérifiées dans la base des faits ?
- La première prémissse **abricot** est vérifiée.
La deuxième prémissse doit être vérifiée ;
- On vérifie s'il existe une règle qui a pour but **pâte** :
Oui, la règle R1.
- On considère maintenant la règle R1 : les prémisses sont vérifiées
- Donc, on peut conclure que le but **tarte aux abricots** est vérifié.

Chainage arrière

Question : Est-ce que je peux faire une tarte aux abricots ?

On cherche une règle qui permet d'obtenir ce but : R4

R4 : abricots + pâte $\rightarrow$ tarte aux abricots

On vérifie : abricots $\rightarrow$ déjà dans les faits

pâte ? Pour avoir pâte, on regarde R1 :

farine, beurre, œufs, sel $\rightarrow$ déjà dans les faits

Donc oui, on peut faire une tarte aux abricots.

Chainage arrière

Méthode	Approche	Objectif
Chaînage avant	Des faits → vers des conclusions	Que puis-je produire ?
Chaînage arrière	Objectif → vérifier si atteignable via règles et faits	Puis-je produire X ?

Conclusion

- Les grandes difficultés rencontrées pendant l'extraction des connaissances des experts puis pendant leur formalisation forment un point faible difficile à contourner.
- Il est possible d'utiliser à la fois des systèmes experts et d'autres techniques comme les réseaux de neurones basées sur des hypothèses plus adaptatives que symboliques (systèmes hybrides).

Chapitre 4: Machine Learning ML

Plan

- Introduction
- Historique
- Principes
- Types d'apprentissage
- Formulation
- Modèles
- Perte
- Etapes d'un projet de ML
- Applications: OPENCV
- Conclusion

Introduction

Le Machine Learning (ML) ou apprentissage automatique est une branche de l'Intelligence Artificielle (IA) qui permet aux machines d'apprendre à partir de données.

Contrairement aux algorithmes classiques, le ML apprend à généraliser sans être explicitement programmé.

Objectif 1: Extraire et exploiter automatiquement l'information contenue dans un jeu de données.

Objectif 2: faire des prédictions ou des décisions automatiques à partir d'expériences (données).

Apprentissage automatique

- ❑ Technique de science des données qui permet aux ordinateurs d'utiliser des données existantes afin de prévoir les tendances, les résultats et les comportements futurs.
- ❑ « Champ d'étude qui donne aux ordinateurs la capacité d'apprendre sans être explicitement programmés » [Arthur Samuel]

Historique

Depuis l'antiquité, le sujet des machines pensantes préoccupe les esprits. Ce concept est la base de pensées pour ce qui deviendra ensuite l'intelligence artificielle, ainsi qu'une de ses sous-branches : l'apprentissage automatique.

Historique

Année	Événement marquant
1950	Alan Turing : test de Turing
1957	Perceptron (Frank Rosenblatt)
1986	Rétropropagation redécouverte
1990	SVM, Random Forests, Boosting
2010	Boom du Deep Learning avec le Big Data et GPU
2012	AlexNet remporte ImageNet avec CNN
2020+	Applications génératives, NLP (ChatGPT, BERT, etc.)

Principes

- Apprentissage par expérience (à partir de données)
- Généralisation : bien prédire sur des données inconnues
- Performance mesurée par des métriques : précision, erreur, etc.

Champs d'application

□ Classification

- La classification consiste à prédire une catégorie ou une étiquette à partir de données.
- Exemple : reconnaître si un e-mail est spam ou non spam, ou déterminer si une image contient un chat ou un chien.
- Elle est utilisée lorsque la variable cible est discrète (ex. : oui/non, type A/B/C)

□ Régression

- La régression vise à prédire une valeur numérique continue à partir de données d'entrée.
- Exemple : prédire le prix d'un logement en fonction de sa surface, du nombre de pièces, etc.
- Elle est utilisée lorsque la variable cible est continue (ex. : température, revenu, temps).

Champs d'application

❑ Détection d'anomalies

- La détection d'anomalies consiste à identifier des comportements inhabituels ou aberrants dans un ensemble de données.
- Exemple : détecter une transaction frauduleuse sur une carte bancaire, ou une défaillance dans un capteur industriel.
- Elle est particulièrement utile dans les domaines de la cybersécurité, de la maintenance prédictive et du contrôle qualité.

❑ Moteurs de recommandation

- Les moteurs de recommandation proposent des contenus ou produits personnalisés à un utilisateur.
- Exemple : les recommandations de films sur Netflix, de produits sur Amazon, ou de vidéos sur YouTube.
- Ils se basent souvent sur les préférences passées, les comportements d'utilisateurs similaires ou le contenu consulté.

Type d'apprentissage

Les différents types d'apprentissage les plus fréquents:

- Apprentissage supervisé
- Apprentissage non supervisé
- Apprentissage semi supervisé
- Apprentissage par renforcement

Type d'apprentissage

Type	Description	Exemple
Supervisé	Données étiquetées	Prédire prix, classier SPAM
Non supervisé	Données non étiquetées	Segmentation clients
Semi-supervisé	Peu de données étiquetées	Traduction automatique
Par renforcement	Apprentissage par récompense	Jeux, robotique

Apprentissage supervisé

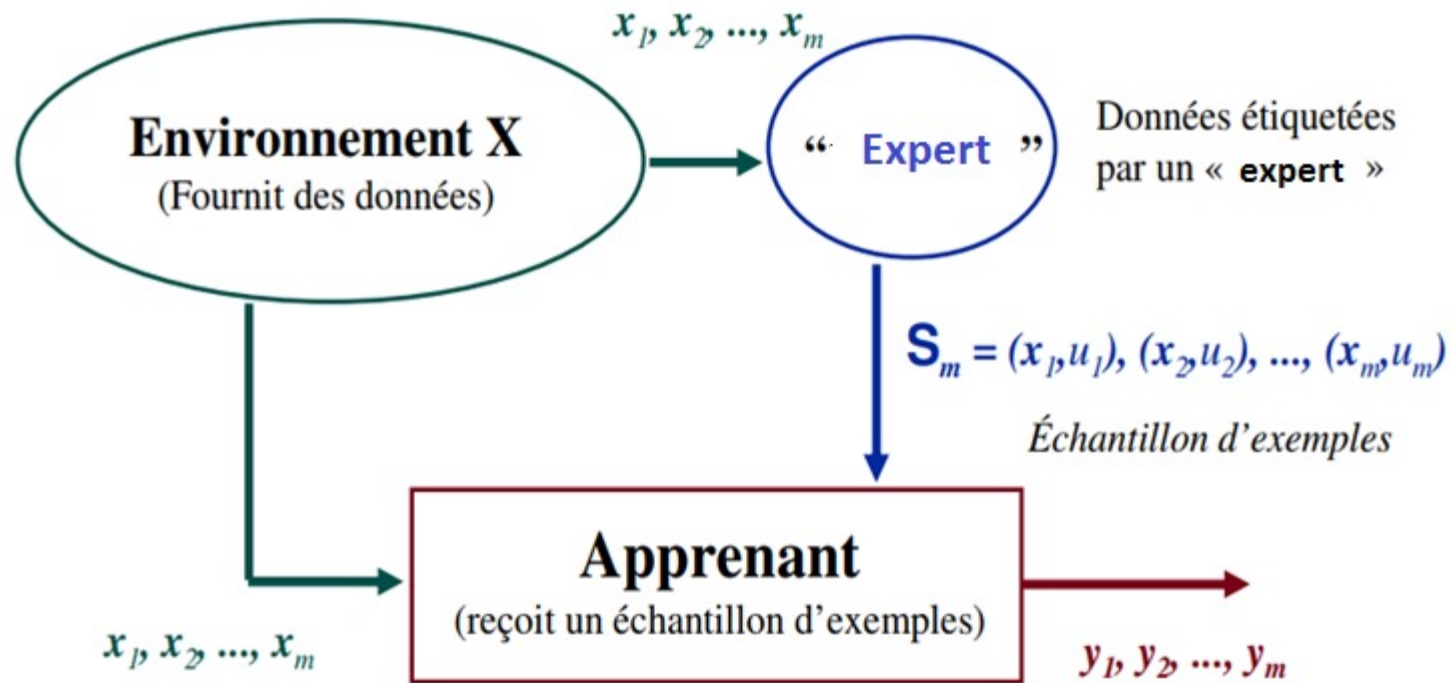
- ❑ Un expert doit préalablement étiqueter des exemples.
- ❑ L'apprenant doit alors trouver ou approximer la fonction qui permet d'affecter la bonne étiquette à ces exemples.
 - Les classes sont prédéterminées et les exemples connus.

Apprentissage supervisé

❑ Le processus se passe en deux phases:

- ❑ 1ère phase (Phase d'*apprentissage*): déterminer un modèle des données étiquetées.
- ❑ 2^{ème} phase (Phase de *test*): Prédire l'étiquette d'une nouvelle donnée, connaissant le modèle préalablement appris.

Apprentissage supervisé



Approximer au mieux la sortie désirée
pour chaque entrée observée

Exemple

Attributs **Classe**



Jour	Ciel	Température	Humidité	Vent	Jouer
J1	Soleil	Chaud	Elevée	Faible	Non
J2	Soleil	Chaud	Elevée	Fort	Non
J3	Couvert	Chaud	Elevée	Faible	Oui
J4	Pluie	Doux	Elevée	Faible	Oui
J5	Pluie	Froid	Normale	Faible	Oui
J6	Pluie	Froid	Normale	Fort	Non
J7	Couvert	Froid	Normale	Fort	Oui
J8	Soleil	Doux	Elevée	Faible	Non
J9	Soleil	Froid	Normale	Faible	Oui
J10	Pluie	Doux	Normale	Faible	Oui
J11	Soleil	Doux	Normale	Fort	Oui
J12	Couvert	Doux	Elevée	Fort	Oui
J13	Couvert	Chaud	Normale	Faible	Oui
J14	Pluie	Doux	Elevée	Fort	Non

Exemple 2

Prix	Surface	Qualité	Adresse postale
313,000	90	3	95000
720,000	110	5	93000
250,000	40	4	44500
290,000	60	3	67000

Exemple 2

Target y

Prix	Surface	Qualité	Adresse postale
313,000	90	3	95000
720,000	110	5	93000
250,000	40	4	44500
290,000	60	3	67000

Exemple 2

Target y

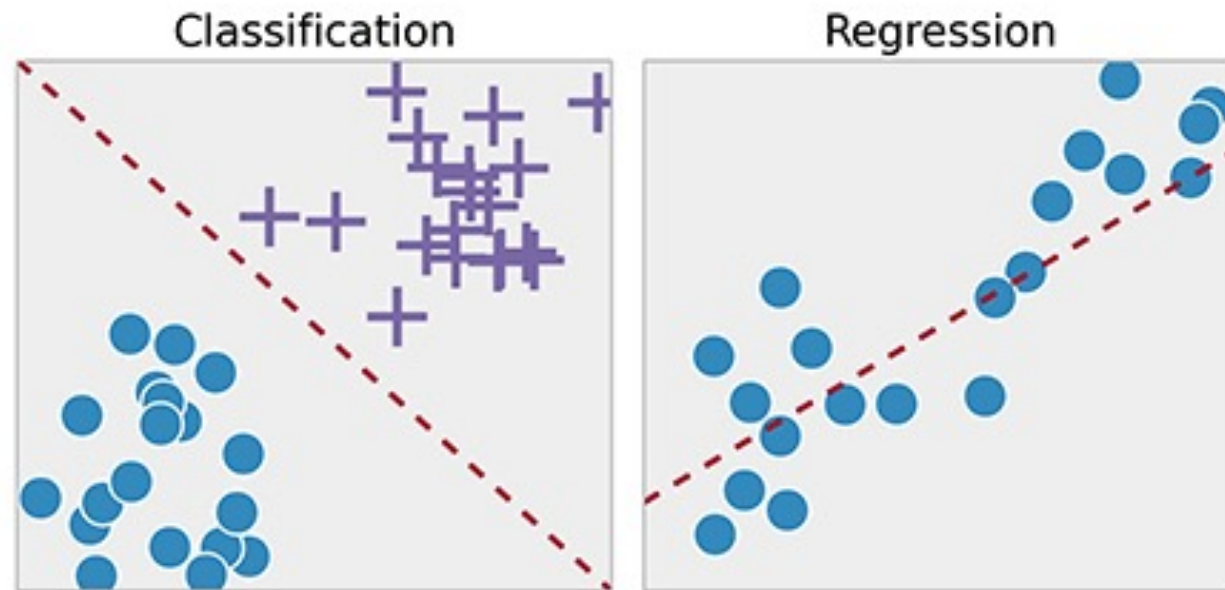
Prix	Surface	Qualité	Adresse postale
313,000	90	3	95000
720,000	110	5	93000
250,000	40	4	44500
290,000	60	3	67000

Exemple 2

Target y	Features		
	x_1	x_2	x_3
Prix	Surface	Qualité	Adresse postale
313,000	90	3	95000
720,000	110	5	93000
250,000	40	4	44500
290,000	60	3	67000

Apprentissage supervisé

- apprendre une fonction de prédiction à partir d'exemples annotés
- deux grandes tâches

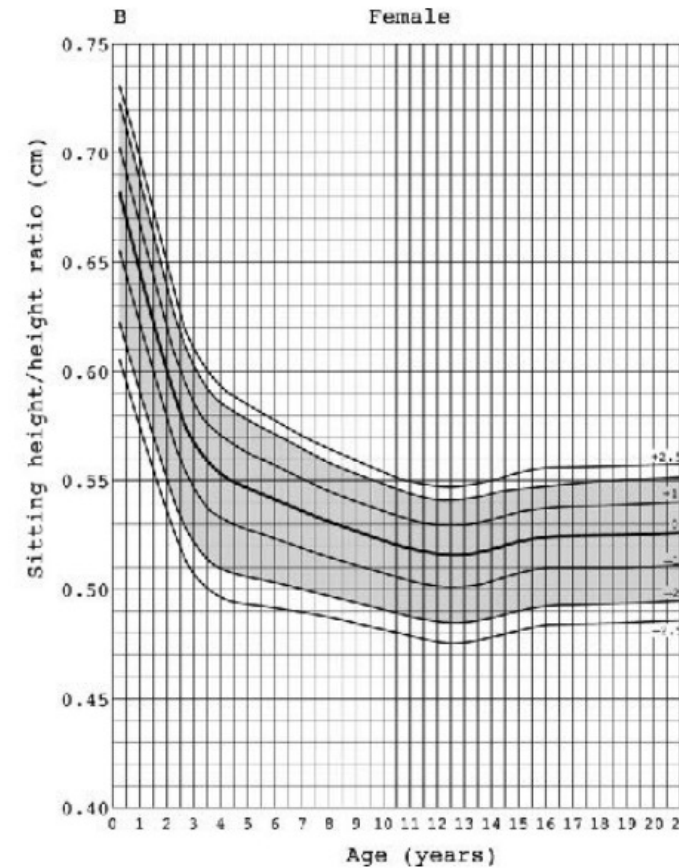


Apprentissage supervisé

Exemples



Classification



Regression

Apprentissage supervisé

Avantages et Inconvénients

Avantages

- Très efficace quand on a suffisamment de données;
- Plus performant que l'homme pour certains problèmes

Inconvénients

- Il faut beaucoup de données;
- Les données doivent être annotées

Apprentissage non supervisé

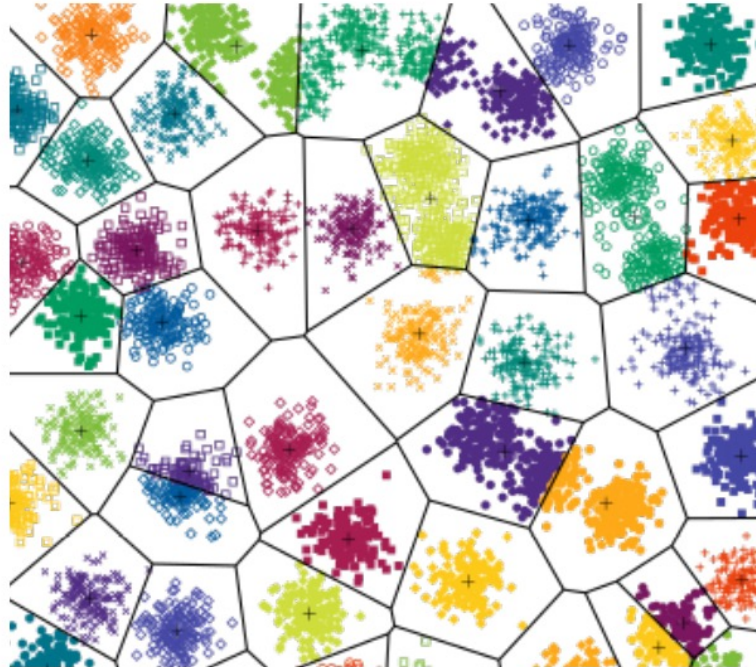
- ❑ Quand le système ou l'opérateur ne disposent que d'exemples, mais non d'étiquettes, et que le nombre de classes et leur nature n'ont pas été prédéterminés, on parle d'apprentissage non supervisé.
- ❑ Aucun expert n'est requis.
- ❑ L'algorithme doit découvrir par lui-même la structure plus ou moins *cachée* des données.

Apprentissage non supervisé

- ❑ Les problèmes d'apprentissage non supervisé les plus populaires sont les analyses de clustering.
- ❑ En cluster analysis, on essaye de séparer les données en **k groupes**.
- ❑ Pour faire ce classement on va optimiser deux choses :
 - Maximiser l'homogénéité des données au sein des groupes, c'est à dire faire en sorte que les observations contenues dans un même groupe se ressemblent le plus possible ;
 - Maximiser l'hétérogénéité entre les groupes, c'est à dire faire en sorte que des observations appartenant à des groupes distincts soient les plus différentes possible.

Apprentissage non supervisé

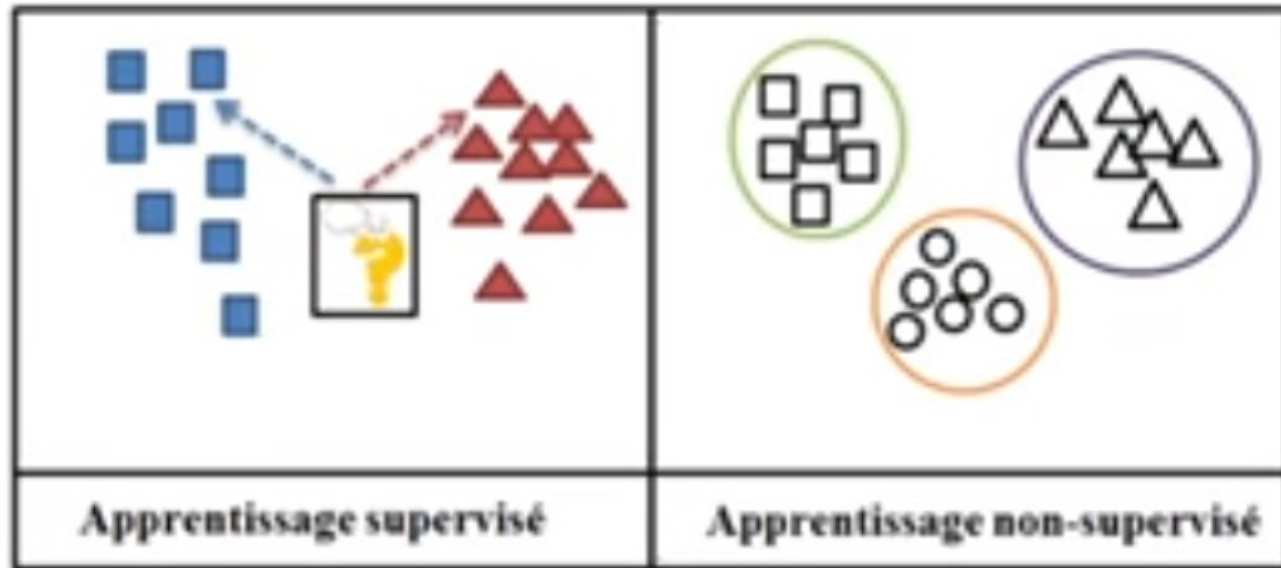
- Regroupe des données similaires, mais sans annotation.
- C'est un problème plus difficile.



Apprentissage non supervisé



Apprentissage supervisé et Apprentissage non supervisé



Apprentissage par renforcement (AR)

❑ L'agent intelligent:

- Observe les effets de ses actions
- Dédit de ses observations la qualité de ses actions
- Améliore ses actions futures

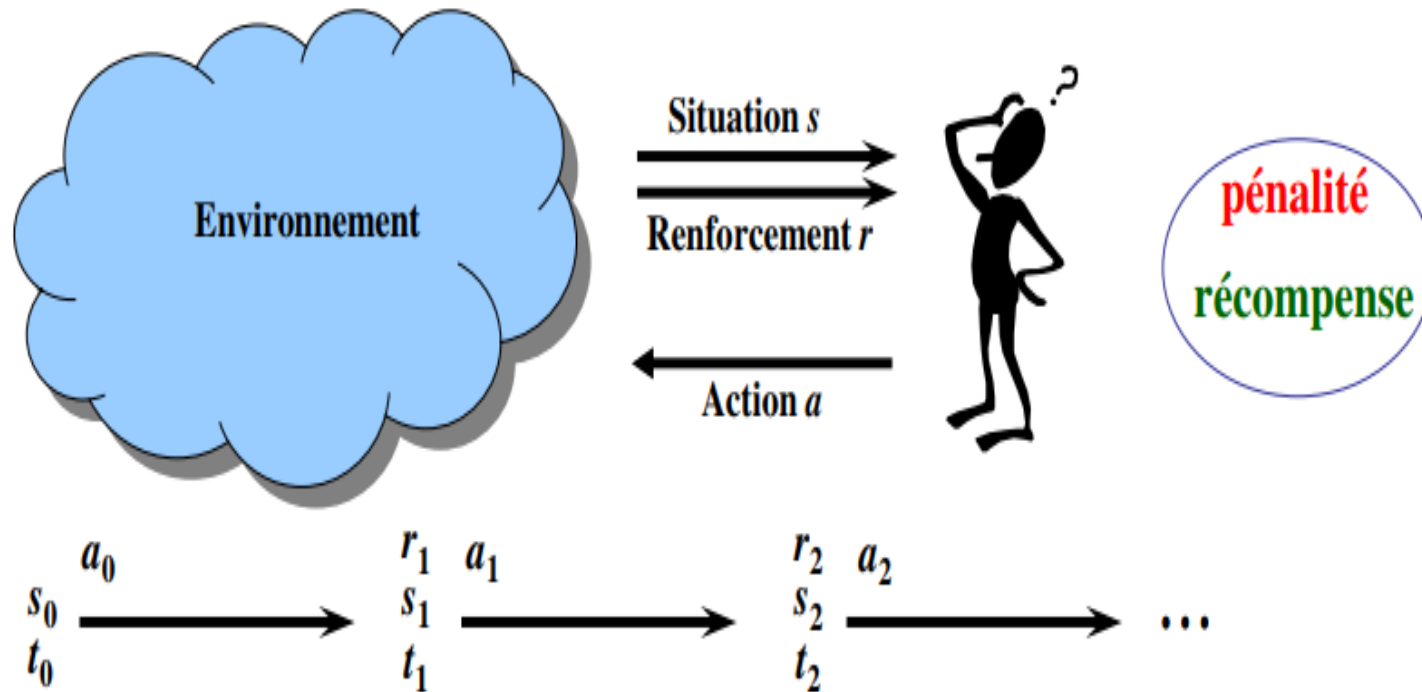
❑ L'action de l'algorithme sur l'environnement produit une valeur de retour qui guide l'algorithme d'apprentissage.

❑ L'agent intelligent décide d'effectuer une action en **fonction de son état** pour interagir avec son environnement. L'environnement lui renvoie un renforcement sous la forme d'une **récompense positive ou négative**. Charge ensuite à l'agent de **maximiser ce renforcement**.

Apprentissage par renforcement (AR)

- ❑ Lorsqu'il y a de l'incertain dans la manière dont l'environnement évolue.
- ❑ L'AR se distingue des autres approches d'apprentissage par plusieurs aspects : L'apprentissage se fait sans supervision Il repose sur le principe d'essai/erreur

Apprentissage par renforcement (AR)



Apprentissage par renforcement

15 MARS 2016 : ALPHA GO BAT LE CHAMPION LEE SEDOL



Formulation

Passons en revue la terminologie de base en matière de ML

Etiquettes

C'est le résultat de la prédiction, notons la variable y .

Caractéristiques

C'est une variable d'entrée, notons la variable x .

$x_1, x_2, \dots, x_n$

Formulation

Exemples

C'est une instance de donnée particulière. Les exemples se répartissent dans deux catégories:

- Exemples étiquetés
- Exemples sans étiquettes

Un exemple étiqueté comprend une ou plusieurs caractéristiques et l'étiquette. Par exemple :

Exemple étiqueté: (x, y)

Modèles

Un modèle définit la relation entre les caractéristiques et l'étiquette.

Il y a deux phases de la durée de vie d'un modèle:

- **Apprentissage**: consiste à créer ou à entraîner le modèle.
exemples étiquetés: (x, y)
- **Inférence**: consiste à appliquer le modèle entraîné à des exemples sans étiquette.
prédictions efficaces : y'

Modèles

Regression et Classification

Les modèles de régression prédisent des valeurs continues.
Exemples: L'âge d'une personne

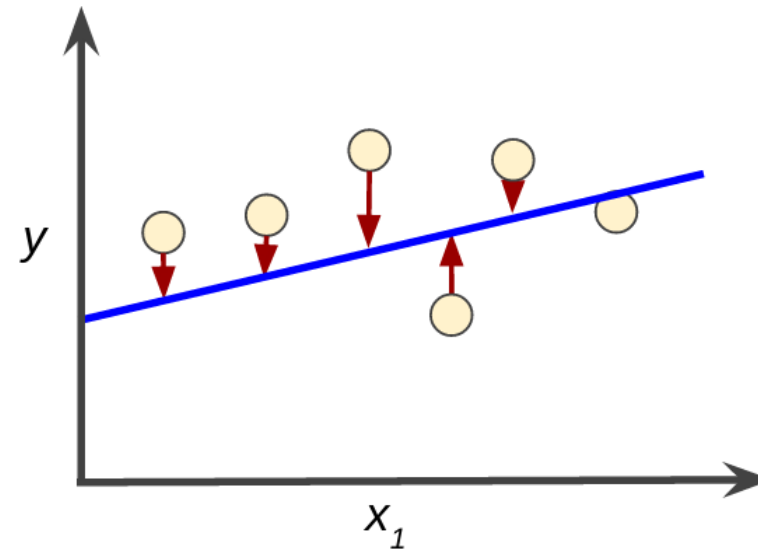
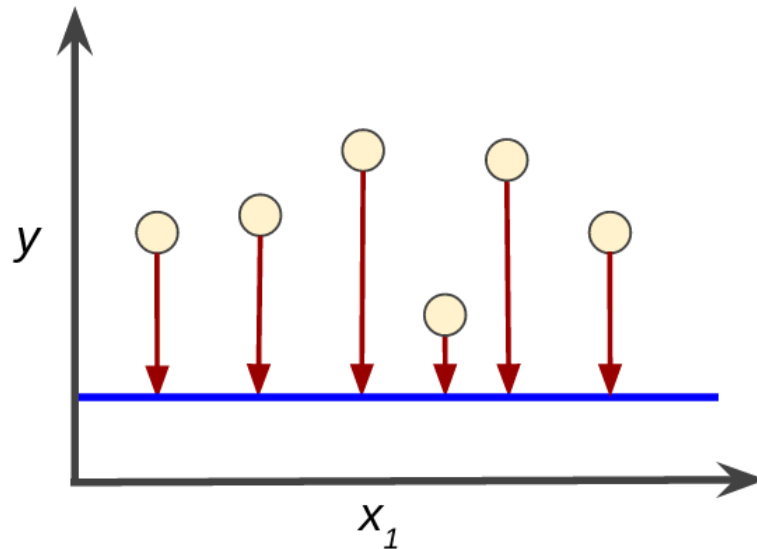
Les modèles de classification prédisent des valeurs discrètes.
Exemples: obstacle routier

Perte

La perte est un nombre qui indique une mauvaise prédiction. Si la prédiction du modèle est parfaite alors la perte est égal à 0.

- Les flèches rouges représentent les pertes.
- La ligne bleue représente les prédictions.

- Les flèches rouges représentent les pertes.
- La ligne bleue représente les prédictions.



Perte

Perte quadratique

La perte quadratique pour les modèles de regression linéaire:

$$\text{perte } L_2 = (y - y')^2$$

Perte

Erreur Quadratique Moyenne(MSE)

L'erreur quadratique moyenne (MSE) correspond à la perte quadratique moyenne pour chaque exemple. Pour calculer l'erreur MSE, il faut additionner toutes les pertes quadratiques de chaque exemple, puis diviser cette somme par le nombre d'exemples

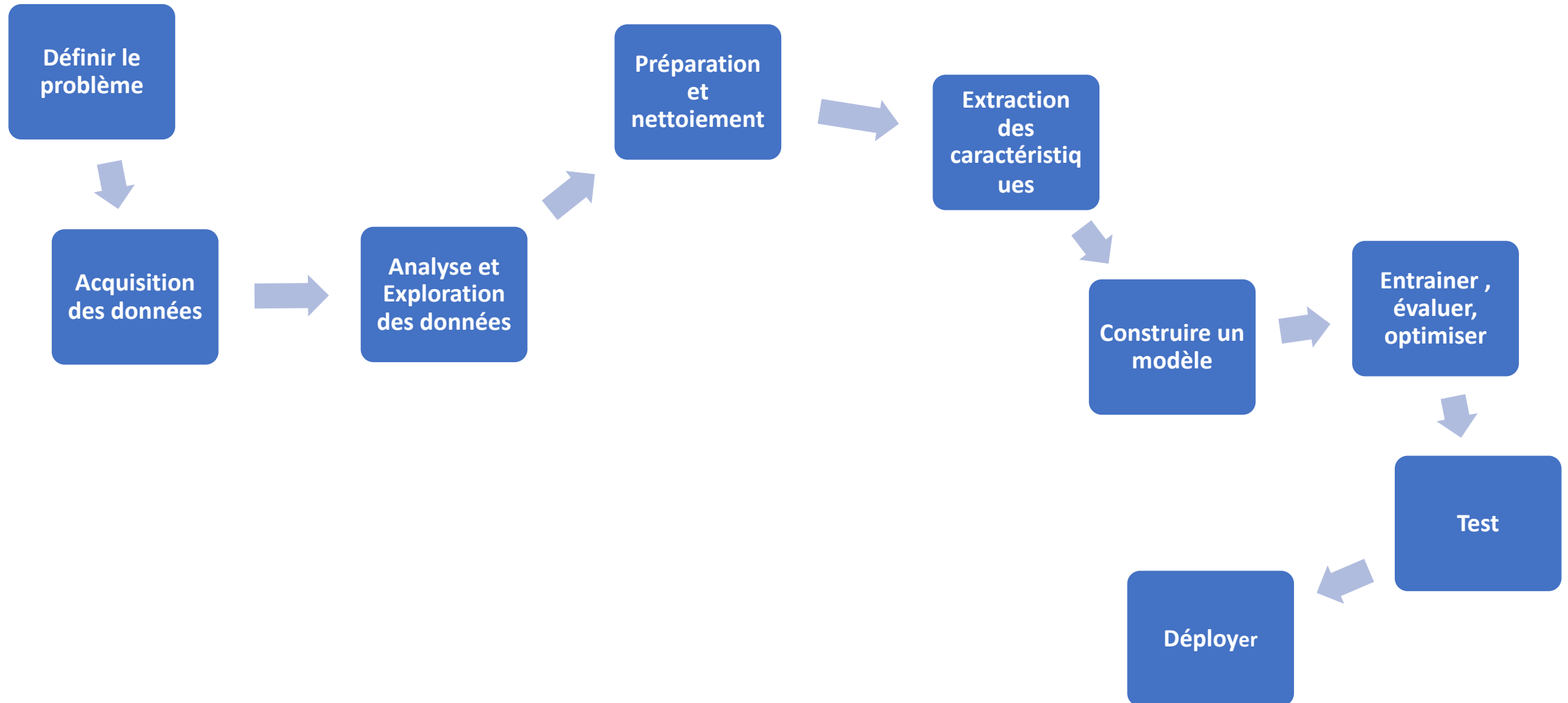
$$MSE = \frac{1}{N} \sum_{(x,y) \in D} (y - prediction(x))^2$$

Perte

Erreur Quadratique Moyenne(MSE)

- (x,y) est un exemple
- $\text{Prediction}(x)$ est une fonction de pondérations et biais
- D est un ensemble de données contenant de nombreux exemples étiquetés
- N est le nombre d'exemples

Etape d'un projet de ML



Application

Application 1: Prédire le prix d'une maison dans une ville

Application 2: Détecter un panneau de signalisation (panneau stop)

Conclusion

Le Machine Learning est au cœur de nombreuses innovations (santé, finance, vision, langage...)

Bien comprendre les bases est crucial pour aborder des concepts plus avancés (Deep Learning, NLP...)

Le ML repose sur la qualité des données, la compréhension du problème, et le choix du modèle adapté

Chapitre 5:

Apprentissage supervisé

Plan

- Introduction
- Regression
- Classification
 - K-NN
 - SVM
- Evaluation des modèles
 - Métriques de performance (précision, rappel, courbe ROC)
- Technique d'amélioration
- Conclusion

Régression linéaire

Définition

La régression linéaire est une technique de modélisation statistique utilisée pour prédire une variable dépendante (ou cible) continue en fonction de une ou plusieurs variables indépendantes (ou features)..

Régression linéaire

Définition

La régression linéaire est une technique statistique utilisée pour déterminer la relation entre les variables. Dans un contexte de ML, la régression linéaire permet de trouver la relation entre les caractéristiques et une étiquette.

Régression linéaire

Définition

L'équation de la régression linéaire est :

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \varepsilon$$

- y : est la variable cible.
- x_i sont les variables indépendantes.
- β_i sont les coefficients de régression.
- ε est l'erreur résiduelle.

Régression linéaire

Objectif

Trouver les coefficients β qui minimisent l'erreur résiduelle entre les valeurs prédites et les valeurs réelles.

Régression linéaire

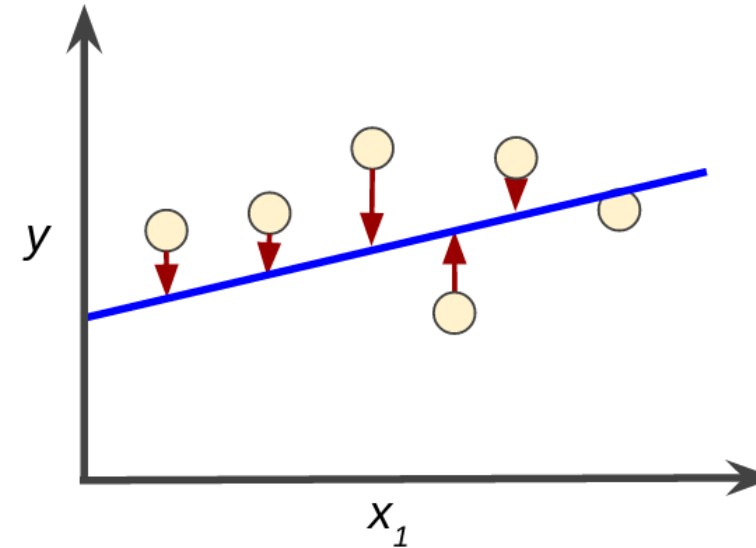
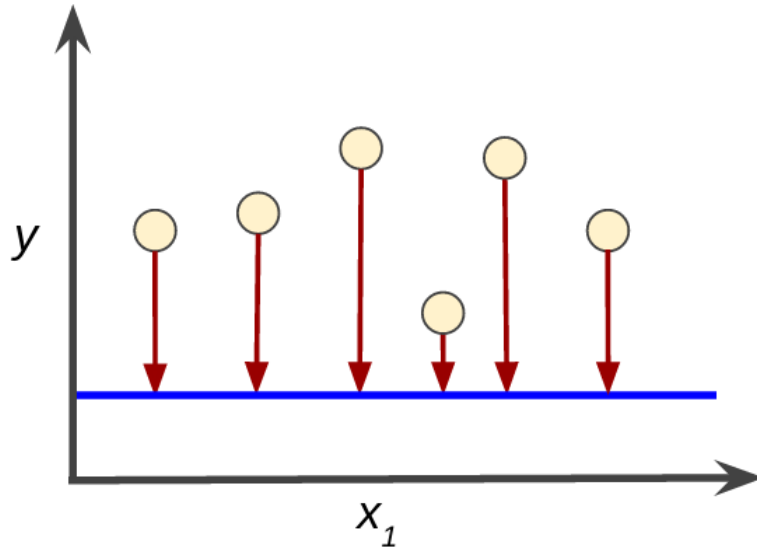
Méthodes / Perte

La perte est une métrique numérique qui décrit l'écart entre les prédictions d'un modèle et la réalité. La perte mesure la distance entre les prédictions du modèle et les étiquettes réelles. L'objectif de l'entraînement d'un modèle est de minimiser la perte, en la réduisant à sa valeur la plus basse possible.

Régression linéaire

Méthodes / Perte

La Perte



Exemple: Valeur réelle 2, valeur prédite 5

$$\text{Perte} = 2 - 5 = -3$$

Régression linéaire

Méthodes / Perte

Voici les deux méthodes les plus courantes pour supprimer le signe:

- Prenez la valeur absolue de la différence entre la valeur réelle et la prédiction.
- Carré la différence entre la valeur réelle et la prédiction.

Régression linéaire

Perte/ Type de perte

Type

Définition

Equation

Perte L_1

Somme des valeurs absolues de la différence entre les valeurs prédites et les valeurs réelles.

$$\sum |valeur\ réelle - valeur\ prédite|$$

Erreur absolue moyenne (EAM)

moyenne (EAM) Moyenne des pertes L_1 sur un ensemble d'exemples $*N*$

$$\frac{1}{N} \sum |valeur\ réelle - valeur\ prédite|$$

Perte L_2

Somme de la différence au carré entre les valeurs prédites et les valeurs réelles.

$$\sum (valeur\ réelle - valeur\ prédite)^2$$

Erreur quadratique moyenne (EAM)

Moyenne des pertes L_2 sur un ensemble d'exemples $*N*$.

$$\frac{1}{N} \sum (valeur\ réelle - valeur\ prédite)^2$$

Régression linéaire

Méthodes

- Gradient Descent

Méthode itérative d'optimisation pour minimiser la fonction de coût.

Mise à jour des coefficients β en fonction du gradient de la fonction de coût.

Régression linéaire

Méthodes

- Gradient Descent

$$\beta_j \leftarrow \beta_j + \alpha \frac{\partial J(\beta)}{\partial \beta_j}$$

α est le taux d'apprentissage, qui détermine la taille des pas de mise à jour.

Classification

Définition

La classification est une technique d'apprentissage supervisé où le modèle est entraîné pour prédire des catégories ou des étiquettes discrètes.

Classification

Concepts clés

- Classes : Les différentes catégories ou étiquettes possibles que le modèle peut prédire.
- Feature Vector : Un ensemble de variables d'entrée (caractéristiques) utilisées pour prédire la classe.
- Frontière de Décision : La limite qui sépare les différentes classes dans l'espace des caractéristiques.
- Surapprentissage (Overfitting) : Un modèle trop ajusté aux données d'entraînement, ce qui réduit sa capacité à généraliser sur de nouvelles données.

Classification

Algorithme: k-plus proches voisins (k-NN)

L'algorithme k-NN classe un point de données en fonction des classes des k

k points de données les plus proches dans l'ensemble d'entraînement. La proximité entre les points est mesurée par une distance (par exemple, la distance euclidienne).

Classification

k-plus proches voisins (k-NN): Étapes de l'Algorithme k-NN

1. Choisir la valeur de k

k est le nombre de voisins les plus proches à considérer pour la classification.

2. Calculer les distances

Pour chaque point de données dans l'ensemble de test, calculer la distance entre ce point et tous les points de l'ensemble d'entraînement. Les distances couramment utilisées sont :

La distance euclidienne: $d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$

Classification

k-plus proches voisins (k-NN): Étapes de l'Algorithme k-NN

3. Trier les distances :

Trier les distances calculées dans l'ordre croissant.

4. Sélectionner les k plus proches voisins :

Sélectionner les k points de l'ensemble d'entraînement ayant les plus petites distances au point de test.

5. Attribuer la classe :

Attribuer la classe majoritaire parmi les k voisins sélectionnés au point de test.

Classification

k-plus proches voisins (k-NN): Exemple

Supposons un ensemble de données où nous devons classer un nouveau point en utilisant k-NN avec $k=3$. Les étapes sont les suivantes :

1. Les données d'entraînement

Point	x_1	x_2	Classe
A	1.0	2.0	Rouge
B	2.0	3.0	Rouge
C	3.0	3.0	Vert
D	6.0	5.0	Vert
E	8.0	8.0	Rouge

Classification

k-plus proches voisins (k-NN): Exemple

2. Point à classifier

Point	x_1	x_2
X	3.5	4.0

3. Calcul des distances (euclidiennes) :

Distance à X

$$\sqrt{(3.5-1.0)^2+(4.0-2.0)^2}=\sqrt{10.25} \approx 3.20$$

$$\sqrt{(3.5-2.0)^2+(4.0-3.0)^2}=\sqrt{3.25} \approx 1.80$$

$$\sqrt{(3.5-3.0)^2+(4.0-3.0)^2}=\sqrt{1.25} \approx 1.12$$

$$\sqrt{(3.5-6.0)^2+(4.0-5.0)^2}=\sqrt{7.25} \approx 2.69$$

$$\sqrt{(3.5-8.0)^2+(4.0-8.0)^2}=\sqrt{36.25} \approx 6.02$$

Classification

k-plus proches voisins (k-NN): Exemple

4. Trier les distances

Point	Distance à X
C	1.12
B	1.80
D	2.69
A	3.20
E	6.02

5. Sélection des $k=3$ plus proches voisins : Points C, B, et D.

Classification

k-plus proches voisins (k-NN): Exemple

4. Trier les distances

Point	Distance à X
C	1.12
B	1.80
D	2.69
A	3.20
E	6.02

5. Sélection des $k=3$ plus proches voisins : Points C, B, et D.

Classification

k-plus proches voisins (k-NN): Exemple

6. Trier le Classes des k voisins :

Point	Classe
C	Vert
B	Rouge
D	Vert

7. Attribuer la classe majoritaire : Vert.

Donc, le point X est classé comme Vert.

Classification

Algorithme: SVM

Les Machines à Vecteurs de Support (SVM) sont des algorithmes de classification puissants et polyvalents qui peuvent être utilisés pour des tâches de classification linéaire et non linéaire.

Classification

SVM: Concepts Clés

1. Hyperplan
2. Marge
3. Vecteur de support
4. Noyaux (kernels)

Classification

SVM: Hyperplan

Un hyperplan est une frontière de décision qui sépare l'espace des caractéristiques en différentes classes

Classification

SVM: Marge

La marge est la distance entre l'hyperplan et les points de données les plus proches de chaque classe, appelés vecteurs de support. Les SVM cherchent à maximiser cette marge.

Classification

SVM: Vecteur de support

Ce sont les points de données qui sont les plus proches de l'hyperplan. Ils déterminent la position et l'orientation de l'hyperplan.

Classification

SVM: Noyau

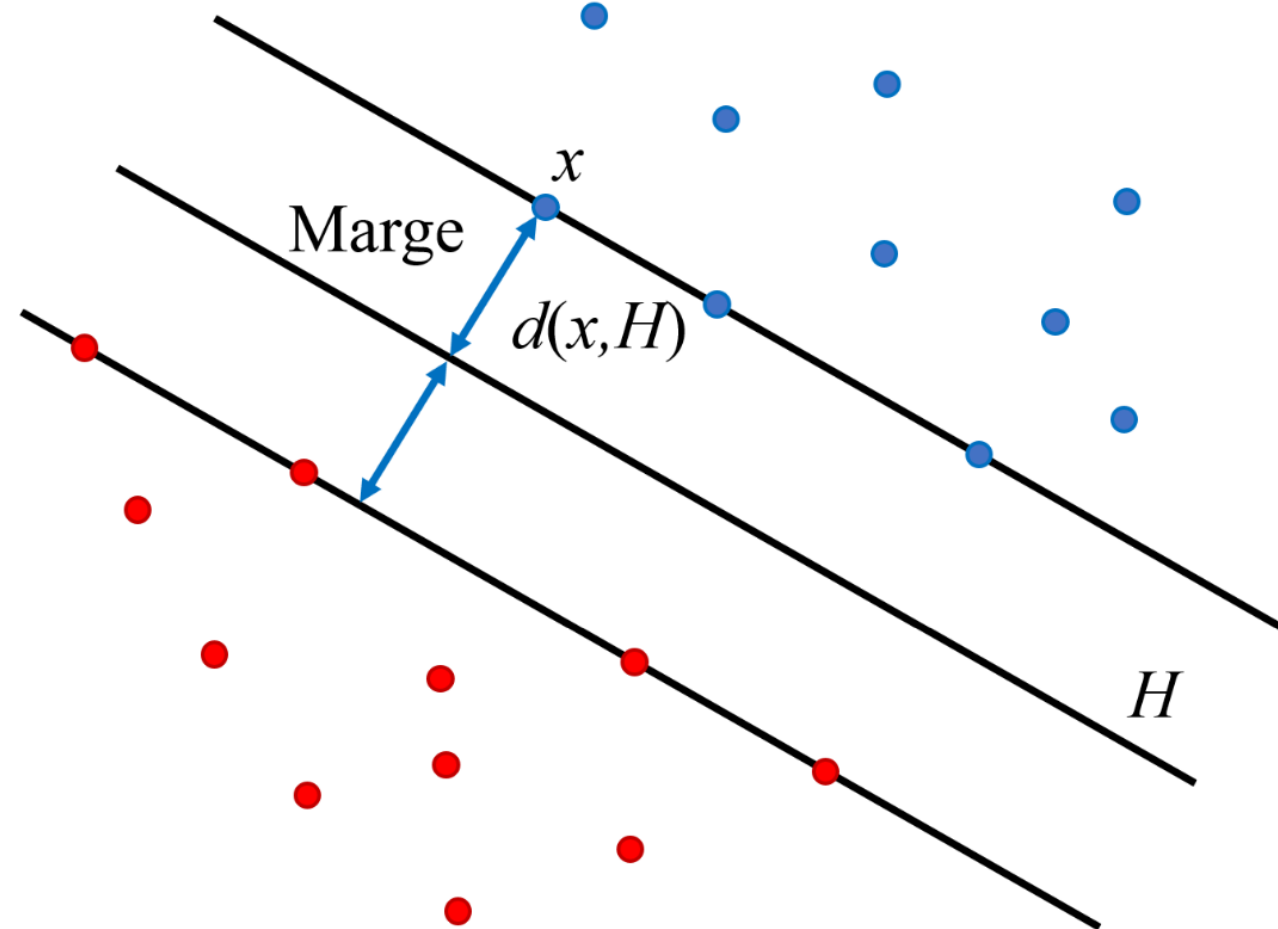
Les SVM peuvent utiliser des fonctions de noyau pour transformer les données dans un espace de dimension supérieure où elles deviennent linéairement séparables.

Classification

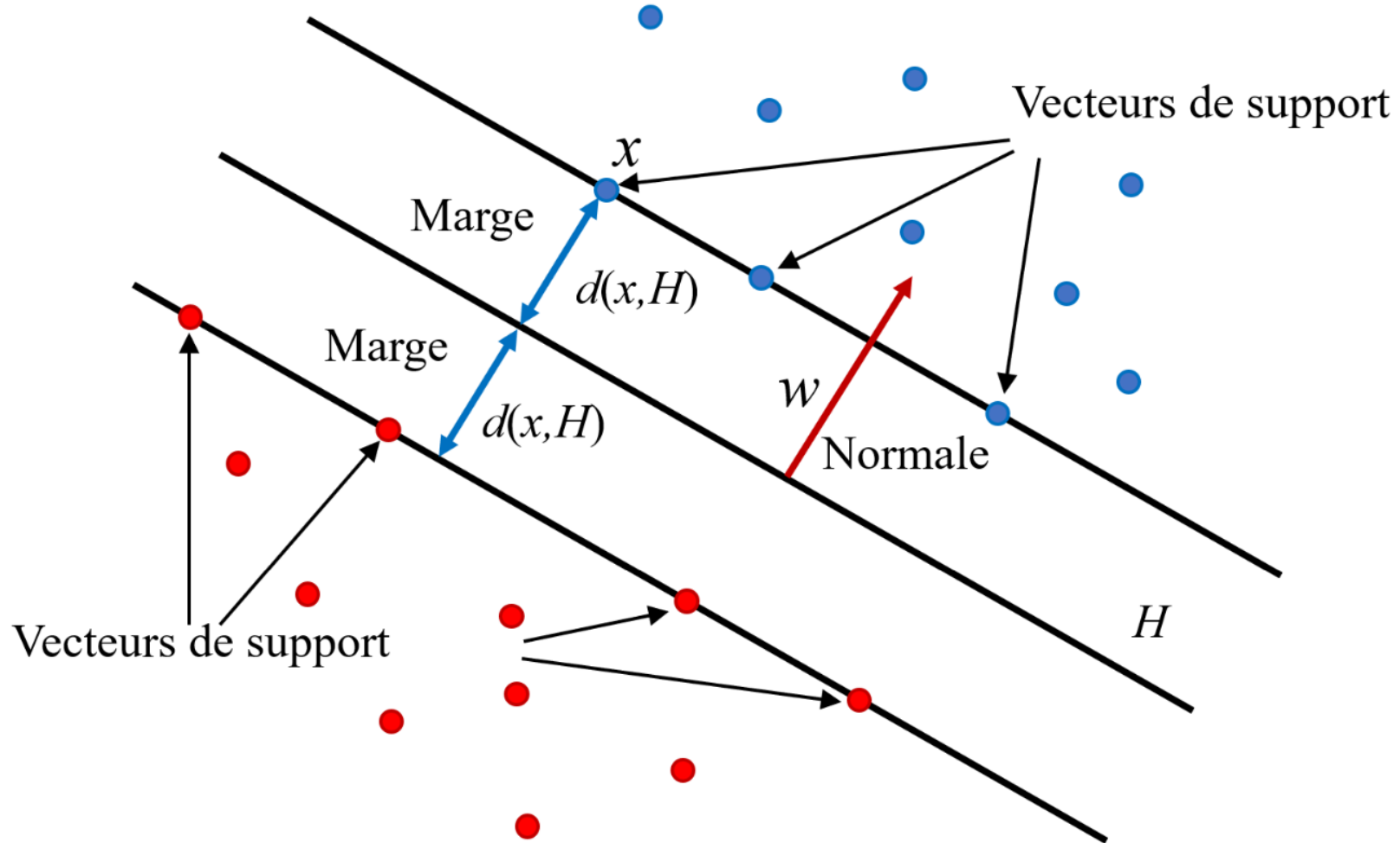
SVM: Noyau

Les SVM peuvent utiliser des fonctions de noyau pour transformer les données dans un espace de dimension supérieure où elles deviennent linéairement séparables.

Classification SVM



Classification SVM



Classification

SVM: Algorithme de Base

1. Définir le problème

Formuler le problème de classification en termes d'optimisation où nous cherchons à maximiser la marge tout en minimisant l'erreur de classification.

2. Trouver l'hyperplan optimal

Trouver l'hyperplan qui maximise la marge entre les classes.

3. Utiliser des noyaux pour la classification non linéaire

1. Appliquer une transformation de noyau pour traiter des données non linéairement séparables.

Classification

SVM: Les types de noyaux

Noyau linéaire

- $K(x, x') = x \cdot x'$
- Utilisé pour les données linéairement séparable

Classification

SVM: Les types de noyaux

Noyau polynomial

- $K(x, x') = (\gamma x \cdot x' + r)^d$
- Peut capturer des relations non linéaires, les hyperparamètres γ , r et d doivent être ajustés

Classification

SVM: Les types de noyaux

Noyau RBF (Radial Basis Function)

- $K(x, x') = \exp(-\gamma \|x - x'\|^2)$
- Très flexible et capable de modéliser des relations complexes. L'hyperparamètre γ doit être ajusté.

Evaluation des modèles

L'évaluation des modèles est une étape cruciale dans le processus de développement des modèles d'apprentissage automatique.

Elle permet de mesurer la performance des modèles sur des données de test, de comparer différents modèles et de choisir le modèle le plus performant

Classification

Méthodes d'évaluation

Séparation des données

- **Train/Test Split** : Diviser les données en deux ensembles, un ensemble d'entraînement pour entraîner le modèle et un ensemble de test pour évaluer le modèle.
- **Validation Croisée (Cross-Validation)** : Diviser les données en k sous-ensembles (folds). Entraîner le modèle sur $k-1$ folds et évaluer sur le fold restant. Répéter ce processus k fois et calculer la performance moyenne.

Classification

Méthodes d'évaluation

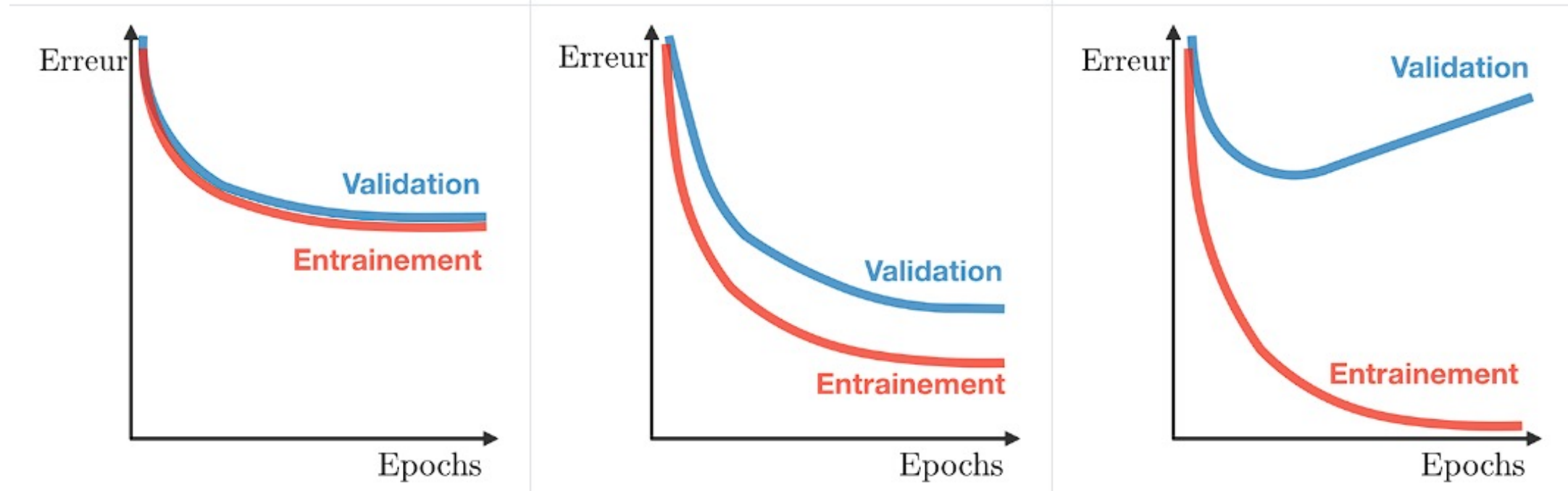
Courbe d'apprentissage

- **Les courbes d'apprentissage montrent la performance du modèle en fonction de la taille de l'ensemble d'entraînement. Elles aident à détecter les problèmes de sous-ajustement (underfitting) et de sur-ajustement (overfitting).**

Classification

Méthodes d'évaluation

Courbe d'apprentissage



Classification

Méthodes d'évaluation

Métrique d'évaluation

1. Exactitude (Accuracy) :

- L'exactitude est la proportion de prédictions correctes sur l'ensemble de test.

$$\text{Exactitude} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Où TP est le nombre de vrais positifs, TN est le nombre de vrais négatifs, FP est le nombre de faux positifs et FN est le nombre de faux négatifs.

Classification

Méthodes d'évaluation

Métrique d'évaluation

2. Précision:

La précision est la proportion de vrais positifs parmi les prédictions positives.

$$Précision = \frac{TP}{TP + FP}$$

Classification

Méthodes d'évaluation

Métrique d'évaluation

3. Rappel (Recall):

Le rappel est la proportion de vrais positifs parmi les cas positifs réels..

$$Recall = \frac{TP}{TP + FN}$$

Classification

Méthodes d'évaluation

Métrique d'évaluation

4. Score F1 (F1 Score) :

Le score F1 est la moyenne harmonique de la précision et du rappel.

$$Score\ F1 = 2 \cdot \frac{Précision * Rappel}{Précision + Rappel}$$

Classification

Méthodes d'évaluation

Métrique d'évaluation

5. Courbe ROC et AUC (Receiver Operating Characteristic and Area Under Curve) :

La courbe ROC trace le taux de vrais positifs (TPR) contre le taux de faux positifs (FPR) pour différents seuils de classification. L'AUC (aire sous la courbe) est une mesure de la capacité du modèle à distinguer entre les classes.

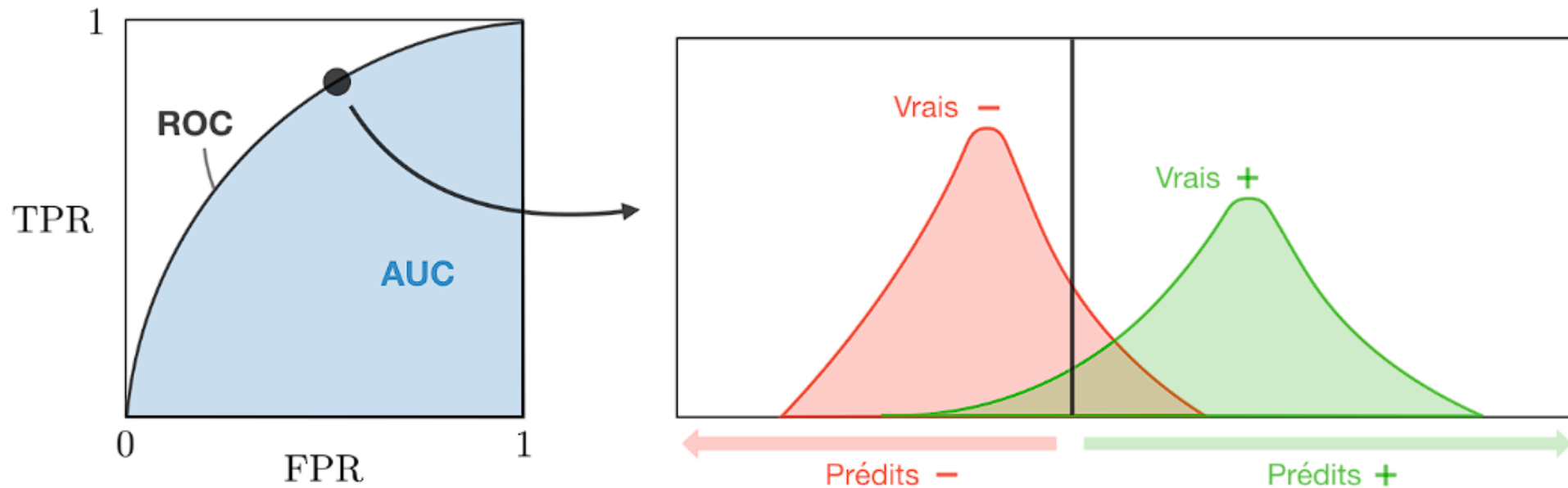
$$TPR = \frac{TP}{TP+FN} \qquad FPR = \frac{FP}{FP+TN}$$

Classification

Méthodes d'évaluation

Métrique d'évaluation

5. Courbe ROC et AUC (Receiver Operating Characteristic and Area Under Curve) :



Classification

Méthodes d'évaluation

Métrique d'évaluation

6. Matrice de Confusion (Confusion Matrix) :

Une matrice de confusion est une table qui permet de visualiser les performances d'un modèle de classification en montrant les véritables étiquettes contre les étiquettes prédites.

Classification

Méthodes d'évaluation

Métrique d'évaluation

6. Matrice de Confusion (Confusion Matrix) :

		True Class	
		Positive	Negative
Predicted Class	Positive	TP	FP
	Negative	FN	TN

Classification

Méthodes d'évaluation

Métrique d'évaluation

Voir TP

Conclusion

Nous avons exploré la régression linéaire et la classification avec les algorithmes k-NN et SVM. L'évaluation des modèles a été réalisée via des méthodes comme la division train/test et la validation croisée, en utilisant des métriques telles que l'exactitude, la précision, le rappel et le score F1. Nous avons aussi discuté des techniques d'amélioration, comme le feature engineering et l'optimisation des hyperparamètres. Ce chapitre fournit une base solide pour développer des modèles efficaces et fiables.